

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

LAB MANUAL

CS23431 – OPERATING SYSTEMS

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Ex. no: 2a)

Shell Script

Aim:

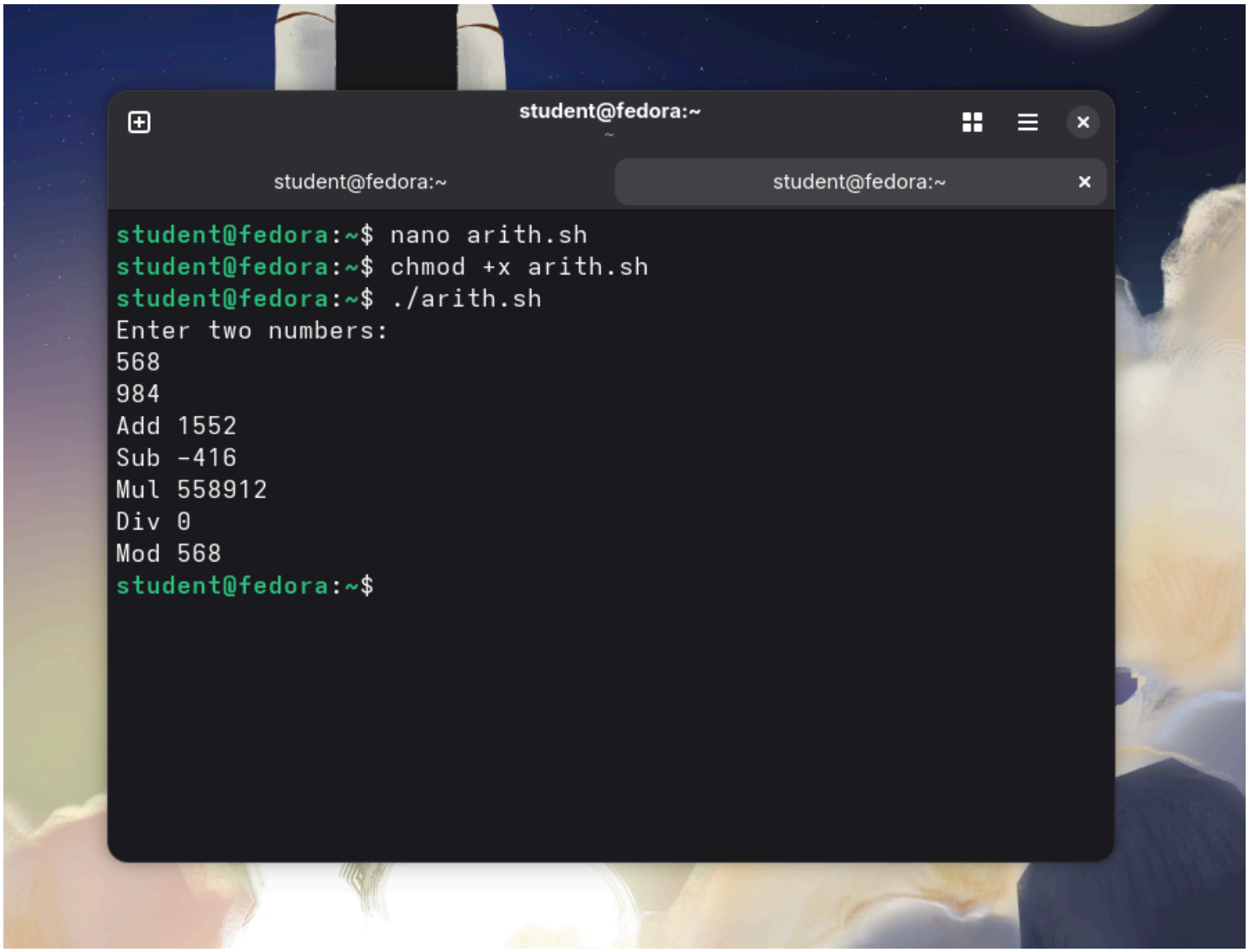
To write a Shellscript to to display basic calculator.

Program:

```
echo enter two no
read a
read b
c=`expr $a + $b`
d=`expr $a - $b`
e=`expr $a \* $b`
f=`expr $a / $b`
g=`expr $a % $b`
echo "add  $c
echo "sub $d"
echo "mul $e"
echo "div $f"
echo "mod $g"
```

OUTPUT:-

```
 [REC@local host~]$sh arith.sh
Enter two no
5
10
add 15
sub -5
mul 50
div 0
mod 5c"
```


student@fedora:~
student@fedora:~
student@fedora:~
student@fedora:~$ nano arith.sh
student@fedora:~$ chmod +x arith.sh
student@fedora:~$ ./arith.sh
Enter two numbers:
568
984
Add 1552
Sub -416
Mul 558912
Div 0
Mod 568
student@fedora:~$

Ex. no: 2b)

Shell Script

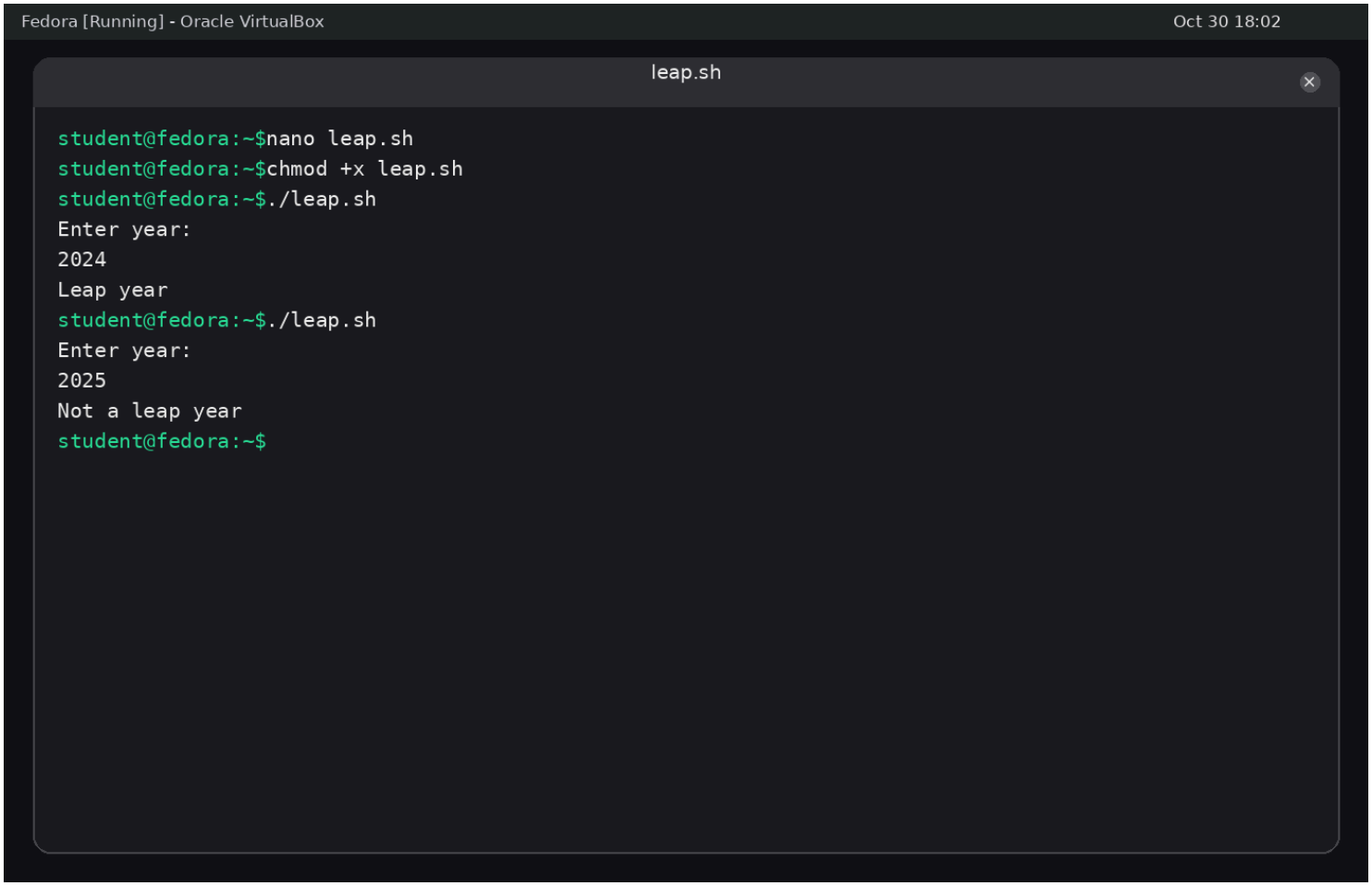
Aim:
To write a Shellscript to test given year is leap or not using conditional statement

Program:

```
echo "enter number"
read a
m=`expr $a % 4`
if [ $m -eq 0 ]
then
echo "leap year "
else
echo "not leap year"
fi
```

OUTPUT:-

```
 [REC @ local host~]$sh leap.sh
enter number
12
leap year
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
leap.sh
student@fedora:~$nano leap.sh
student@fedora:~$chmod +x leap.sh
student@fedora:~$./leap.sh
Enter year:
2024
Leap year
student@fedora:~$./leap.sh
Enter year:
2025
Not a leap year
student@fedora:~$

Ex. No.: 3a)

Shell Script – Reverse of Digit

Aim:

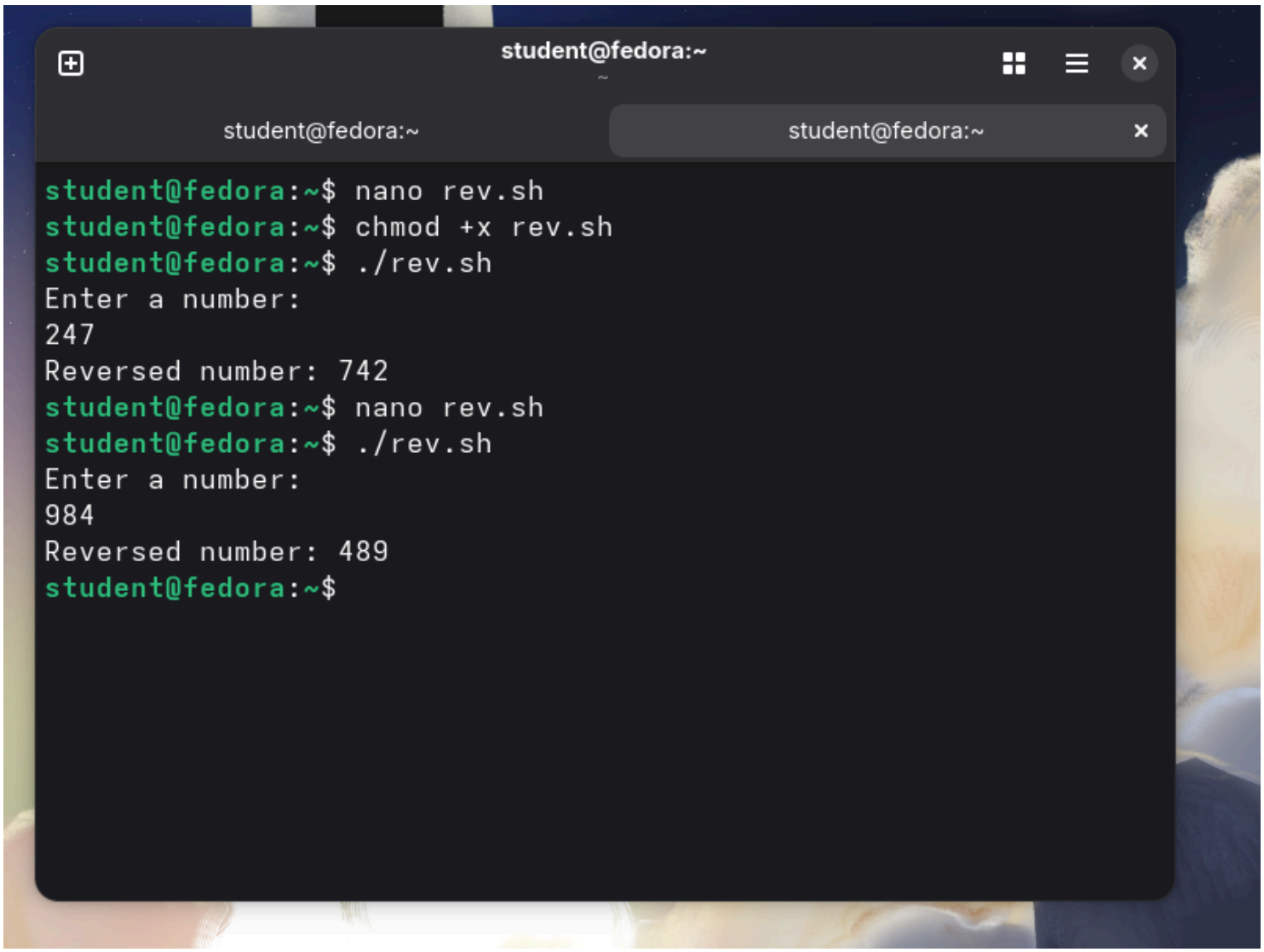
To write a Shellscript to reverse a given digit using looping statement.

Program:

```
echo " enter number "
read n
rev=0
while [ $n -gt 0 ]
do
d=`expr $n % 10`
rev=$(echo "$rev * 10 + $d"|bc)
n=`expr $n / 10`
done
echo $rev
```

OUTPUT:-

```
 [REC@local host~]$sh indhu.sh
enter number
123
321
```


student@fedora:~
student@fedora:~
student@fedora:~
student@fedora:~$ nano rev.sh
student@fedora:~$ chmod +x rev.sh
student@fedora:~$ ./rev.sh
Enter a number:
247
Reversed number: 742
student@fedora:~$ nano rev.sh
student@fedora:~$ ./rev.sh
Enter a number:
984
Reversed number: 489
student@fedora:~$

Ex. No.: 3b)

Shell Script – Reverse of Digit

Aim:

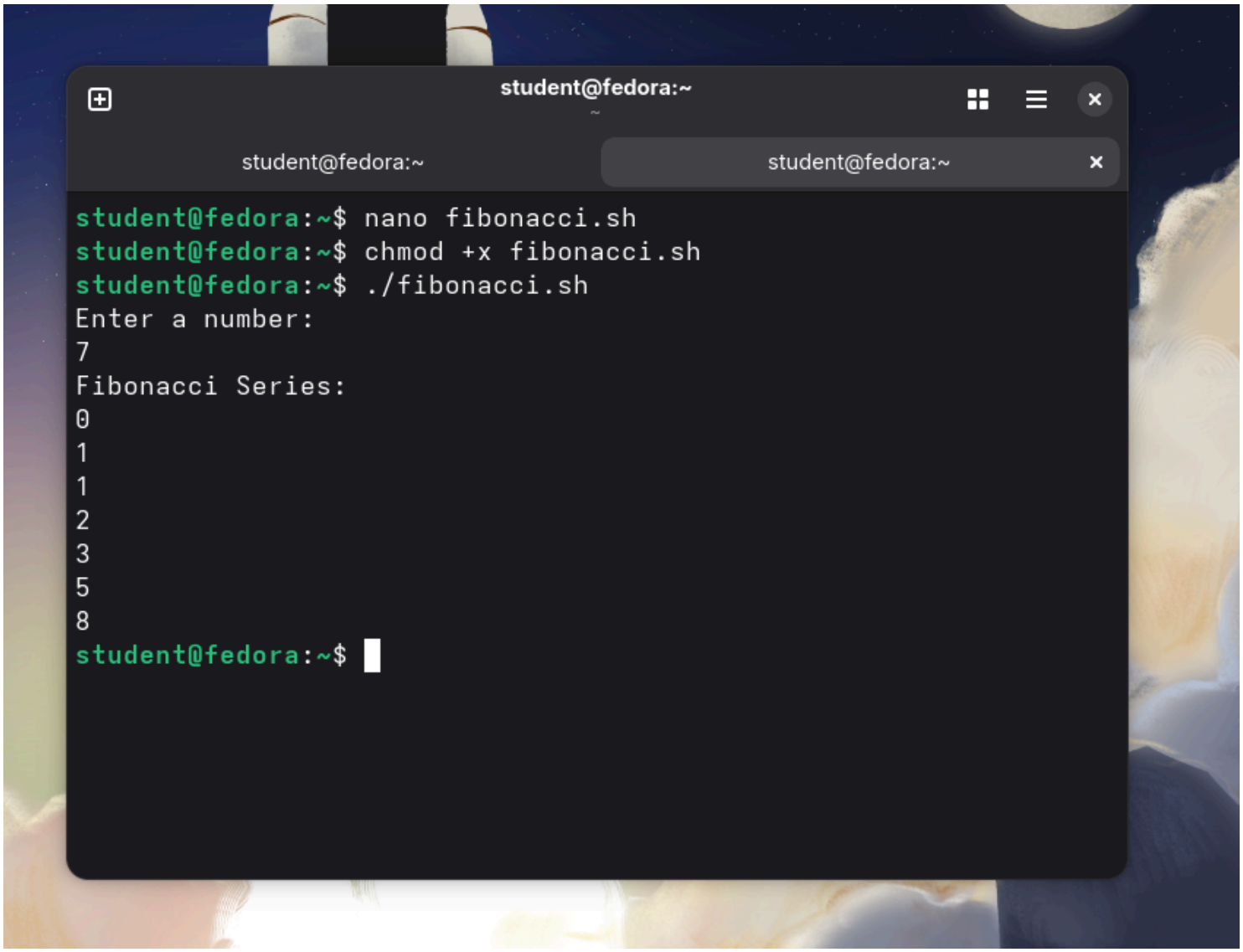
To write a Shellscript to generate a Fibonacci series using for loop.

Program:

```
echo  "enter number"
read n
a=-1
b=1
echo "fibonacci series"
for((i=1;i<=$n;i++))
do
c=$(echo "$a + $b"|bc)
echo $c
a=$b
b=$c
done
```

OUTPUT:-

```
 [REC@local host~]$sh indhu.sh
enter number
21
fibonacci series
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
```


student@fedora:~
student@fedora:~
student@fedora:~
student@fedora:~$ nano fibonacci.sh
student@fedora:~$ chmod +x fibonacci.sh
student@fedora:~$ ./fibonacci.sh
Enter a number:
7
Fibonacci Series:
0
1
1
2
3
5
8
student@fedora:~$

Ex. No.: 4a)

EMPLOYEE AVERAGE PAY

Aim:

To find out the average pay of all employees whose salary is more than 6000 and no. of days worked is more than 4.

Algorithm:

1. Create a flat file emp.dat for employees with their name, salary per day and number of days worked and save it.
2. Create an awk script emp.awk
3. For each employee record do
 a. If Salary is greater than 6000 and number of days worked is more than 4then print name and salary earned
 b. Compute total pay of employee
4. Print the total number of employees satisfying the criteria and their average pay.

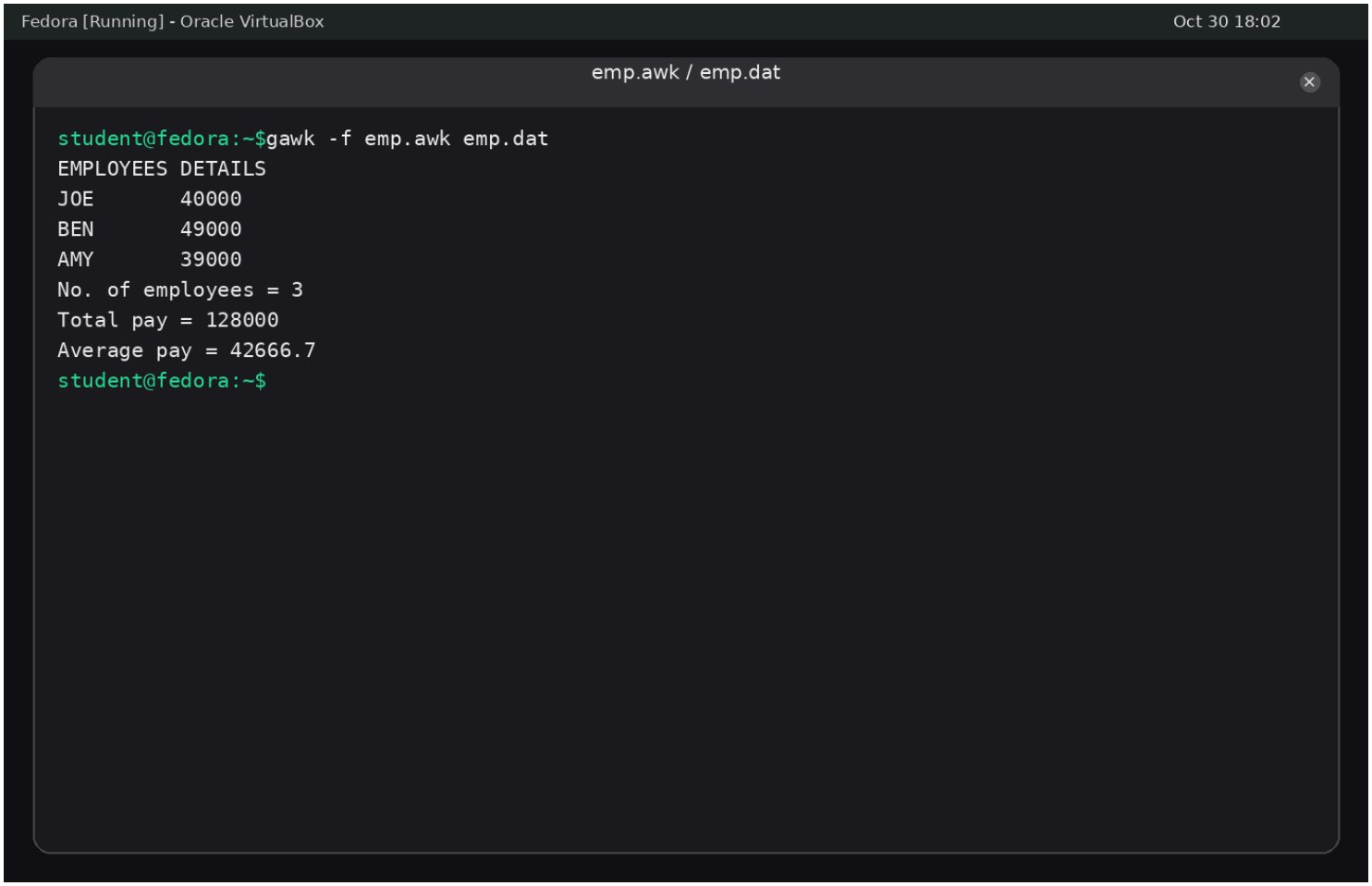
Program Code:

```
//emp.awk
BEGIN{print "EMPLOYEES DETAILS"}
 {#salary should be greater than 6000 and days
 more than 4
 if($2>6000 && $3>4)
 {
 print $1,"\t\t", $2*$3
 pay=pay+ $2*$3
 count=count+1
 }
 }

 END{
 {#action part
 print "no of employees are=",NR/count+1
 print "total pay=",pay
 print "average pay=",pay/count
 }
 }
```

```
//emp.dat – Col1 is name, Col2 is Salary Per Day and Col3 is //no. of days worked JOE
8000 5
RAM 6000 5


TIM 5000 6
BEN 7000 7
AMY 6500 6
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
emp.awk / emp.dat
student@fedora:~$gawk -f emp.awk emp.dat
EMPLOYEES DETAILS
JOE 40000
BEN 49000
AMY 39000
No. of employees = 3
Total pay = 128000
Average pay = 42666.7
student@fedora:~$

Ex. No.: 4b)

RESULTS OF EXAMINATION

Aim:

To print the pass/fail status of a student in a class.

Algorithm:

1. Read the data from file

2. Get a data from each column

3. Compare the all subject marks column

   a. If marks less than 45 then print Fail

   b. else print Pass

Program Code:

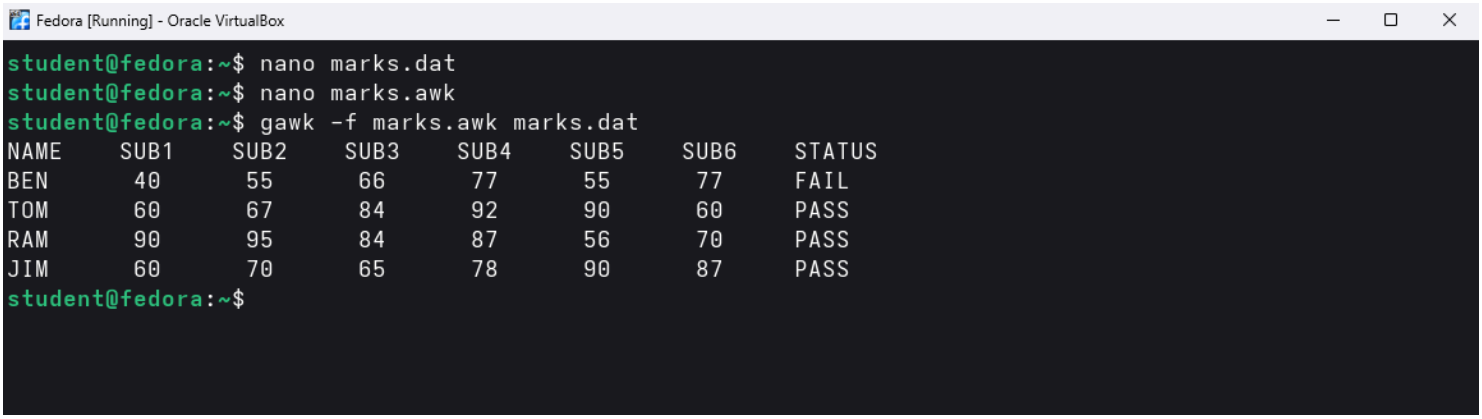
//marks.dat

```
//Col1- name, Col 2 to Col7 – marks in various subjects
BEN 40 55 66 77 55 77
TOM 60 67 84 92 90 60
RAM 90 95 84 87 56 70
JIM 60 70 65 78 90 87
```

//marks.awk

```
BEGIN{
        print "NAME", "\t","SUB-1","\t","SUB-2","\t","SUB- 3","\t","SUB-4","\t","SUB
        5","\t","SUB-6","\t","STATUS"
        print"_____\n"  }
{ #BODY
        if ( $2 < 45 || $3 < 45 || $4 < 45 || $5 < 45 || $6 < 45
         || $7 < 45)
         {
print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",
                $6,"\t", $7,"\t","FAIL"
}
        else
        {


print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",
$6,"\t",$7,"\t","PASS"
        }
}
```


Fedora [Running] - Oracle VirtualBox
student@fedora:~$ nano marks.dat
student@fedora:~$ nano marks.awk
student@fedora:~$ gawk -f marks.awk marks.dat
NAME SUB1 SUB2 SUB3 SUB4 SUB5 SUB6 STATUS
BEN 40 55 66 77 55 77 FAIL
TOM 60 67 84 92 90 60 PASS
RAM 90 95 84 87 56 70 PASS
JIM 60 70 65 78 90 87 PASS
student@fedora:~$

Ex. No.: 5

System Calls Programming

Aim: To experiment system calls using fork(), execlp() and pid() functions.

Algorithm:

1. **Start**
   - Include the required header files (`stdio.h` and `stdlib.h`).
2. **Variable Declaration**
   - Declare an integer variable `pid` to hold the process ID.
3. **Create a Process**
   - Call the `fork()` function to create a new process. Store the return value in the `pid` variable:
     - If `fork()` returns:
       - `-1`: Forking failed (child process not created).
       - `0`: Process is the child process.
       - Positive integer: Process is the parent process.
4. **Print Statement Executed Twice**
   - Print the statement:

     ```
     scss
     Copy code
     THIS LINE EXECUTED TWICE
     ```

     (This line is executed by both parent and child processes after `fork()`).

5. **Check for Process Creation Failure**
   - If `pid == -1`:
     - Print:

       ```
       Copy code
       CHILD PROCESS NOT CREATED
       ```

     - Exit the program using `exit(0)`.
6. **Child Process Execution**
   - If `pid == 0` (child process):
     - Print:
       - Process ID of the child process using `getpid()`.
       - Parent process ID of the child process using `getppid()`.
7. **Parent Process Execution**
   - If `pid > 0` (parent process):
     - Print:
       - Process ID of the parent process using `getpid()`.
       - Parent's parent process ID using `getppid()`.

8. **Final Print Statement**
    - Print the statement:

```
objectivec
Copy code
IT CAN BE EXECUTED TWICE
```

(This line is executed by both parent and child processes).

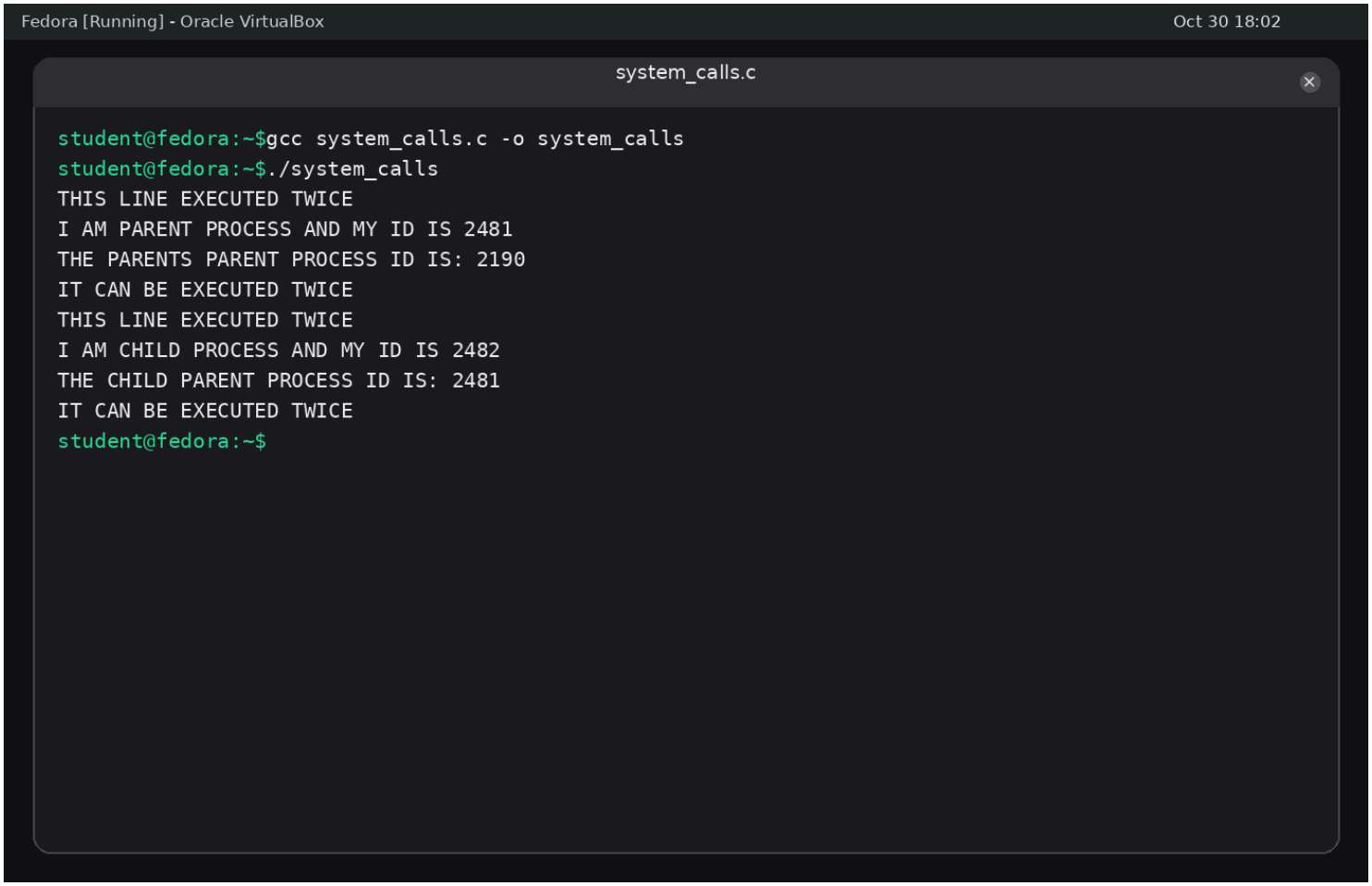
9. **End**

Program:

FORK SYSTEM CALL PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
int main( )
{
int pid;
pid=fork( );
printf("\n THIS LINE EXECUTED TWICE");
if(pid==-1) {
printf("\n CHILD PROCESS NOT CREATED\n");

exit(0);
}
if(pid==0) {
printf("\n I AM CHILD PROCESS AND MY ID IS %d \n",getpid( ));
printf("\n THE CHILD PARENT PROCESS ID IS:%d \n",getppid( ));
}
else
{
printf("\n I AM PARENT PROCESS AND MY ID IS:%d\n",getpid( ));
printf("\n THE PARENTS PARENT PROCESS ID IS:%d\n",getppid( ));
}
printf("\n IT CAN BE EXECUTED TWICE");
printf("\n");
}
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
system_calls.c
student@fedora:~$gcc system_calls.c -o system_calls
student@fedora:~$./system_calls
THIS LINE EXECUTED TWICE
I AM PARENT PROCESS AND MY ID IS 2481
THE PARENTS PARENT PROCESS ID IS: 2190
IT CAN BE EXECUTED TWICE
THIS LINE EXECUTED TWICE
I AM CHILD PROCESS AND MY ID IS 2482
THE CHILD PARENT PROCESS ID IS: 2481
IT CAN BE EXECUTED TWICE
student@fedora:~$

Ex. No.: 6a)

FIRST COME FIRST SERVE

Aim:

To implement First-come First- serve(FCFS) scheduling technique

Algorithm:

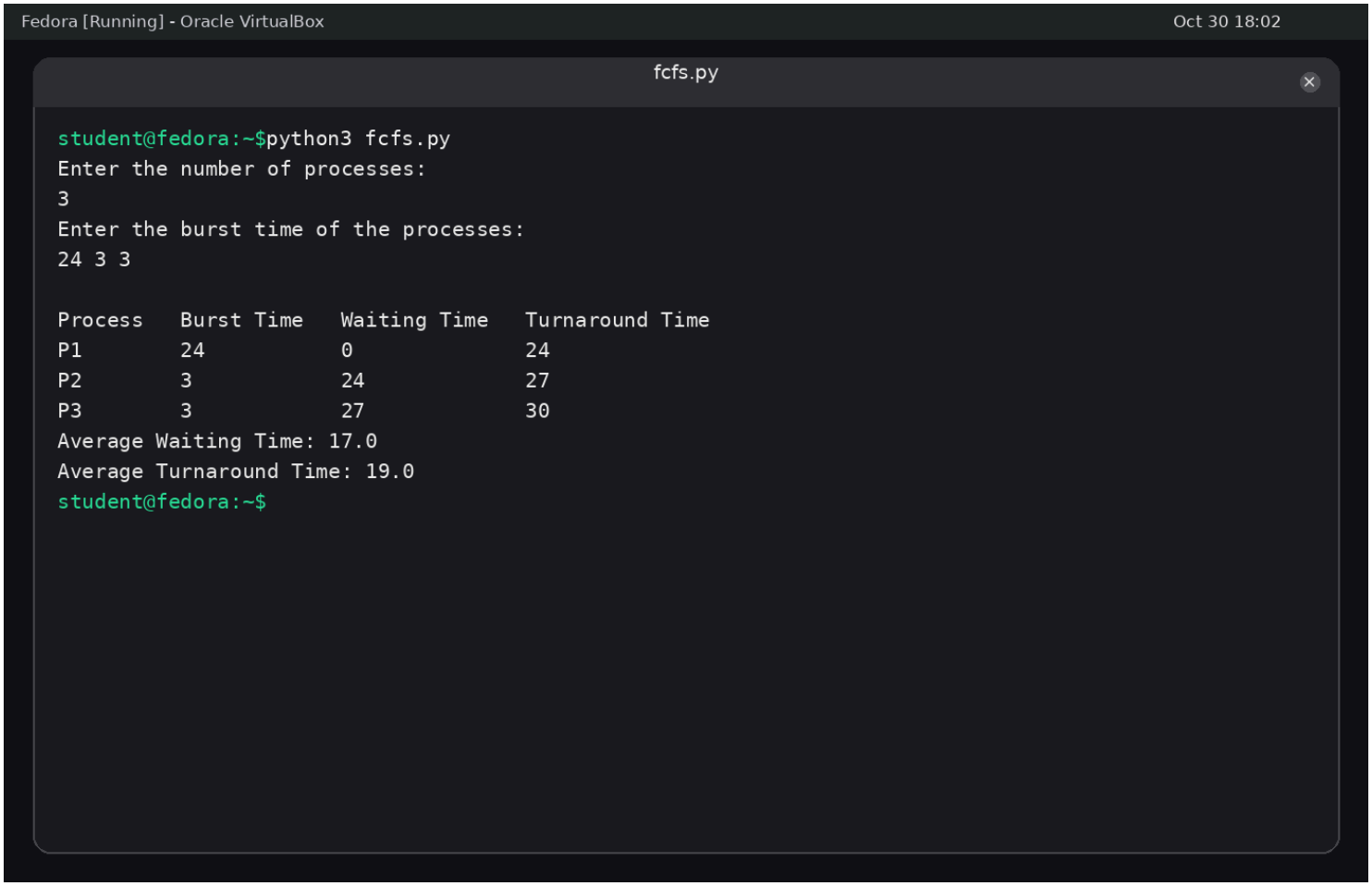
1. Get the number of processes from the user.
2. Read the process name and burst time.
3. Calculate the total process time.
4. Calculate the total waiting time and total turnaround time for each process 5. Display the process name & burst time for each process. 6. Display the total waiting time, average waiting time, turnaround time

Program Code:

```
bt=[]
print("Enter the number of process: ")
n=int(input())
print("Enter the burst time of the processes: \n")
bt=list(map(int, input().split()))
wt=[]
avgwt=0
tat=[]
avgtat=0
wt.insert(0,0)
tat.insert(0,bt[0])
for i in range(1,len(bt)):
        wt.insert(i,wt[i-1]+bt[i-1])
        tat.insert(i,wt[i]+bt[i])
        avgwt+=wt[i]
        avgtat+=tat[i]
avgwt=float(avgwt)/n
avgtat=float(avgtat)/n
print("\n")
print("Process\t Burst Time\t Waiting Time\t Turnaround
Time")
for i in range(0,n):
        print(str(i)+"\t\t"+str(bt[i])+"\t\t"+str(wt[i])+"\t
 \t"+str(tat[i]))
        print("\n")
print("Average Waiting time is: "+str(avgwt))
print("Average Turn Around Time is: "+str(avgtat))
```

Output:

```
Enter the number of process:
3
Enter the burst time of the processes:
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
fcfs.py
student@fedora:~$python3 fcfs.py
Enter the number of processes:
3
Enter the burst time of the processes:
24 3 3
Process Burst Time Waiting Time Turnaround Time
P1 24 0 24
P2 3 24 27
P3 3 27 30
Average Waiting Time: 17.0
Average Turnaround Time: 19.0
student@fedora:~$

Ex. No.: 6b)

SHORTEST JOB FIRST

Aim:

To implement the Shortest Job First(SJF) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes as input from the user.
3. Read the process name, arrival time and burst time
4. Initialize waiting time, turnaround time & flag of read processes to zero. 5. Sort based on burst time of all processes in ascending order 6. Calculate the waiting time and turnaround time for each process. 7. Calculate the average waiting time and average turnaround time. 8. Display the results.

Program Code:

```
bt=[] #bt stands for burst time
print("Enter the number of process: ")
n=int(input())
processes=[]
for i in range(0,n):
 processes.insert(i,i+1)
print("Enter the burst time of the processes: \n")
bt=list(map(int, raw_input().split()))
for i in range(0,len(bt)-1): #applying bubble sort on bt
 for j in range(0,len(bt)-i-1):
 if(bt[j]>bt[j+1]):
 temp=bt[j]
 bt[j]=bt[j+1]
 bt[j+1]=temp
 temp=processes[j]
 processes[j]=processes[j+1]
 processes[j+1]=temp
wt=[] #wt stands for waiting time
avgwt=0 #average of waiting time
tat=[] #tat stands for turnaround time
avgtat=0 #average of total turnaround time
wt.insert(0,0)
tat.insert(0,bt[0])
for i in range(1,len(bt)):
wt.insert(i,wt[i-1]+bt[i-1])
tat.insert(i,wt[i]+bt[i])
avgwt+=wt[i]
avgtat+=tat[i]
avgwt=float(avgwt)/n
avgtat=float(avgtat)/n
print("\n")
```

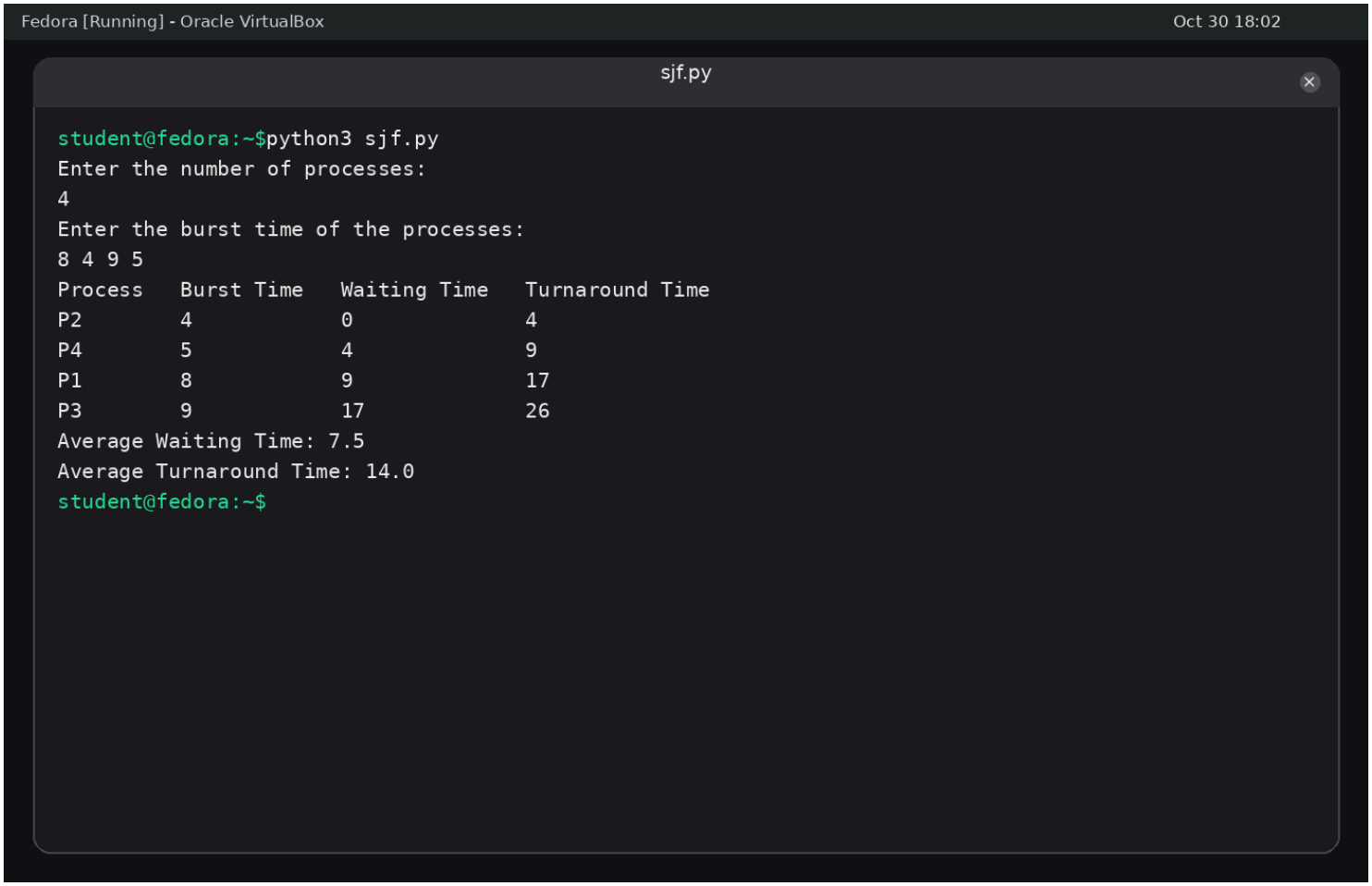
```
print("Process\t Burst Time\t Waiting Time\t Turn Around
Time")
for i in range(0,n):
 print(str(processes[i])+"\t\t"+str(bt[i])+"\t\t"+str(wt[i])
+"\t\t"+str(tat[i]))
print("Average Waiting time is: "+str(avgwt))
print("Average Turn Around Time is: "+str(avgtat))
```

Output:

```
Enter the number of process:
4
Enter the burst time of the processes:
8 4 9 5

Process Burst Time Waiting Time Turn Around Time  2 4 0 4
4 5 4 9
1 8 9 17
3 9 17 26

Average Waiting time is: 7.5
Average Turn Around Time is: 13.0
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
sjf.py
student@fedora:~$python3 sjf.py
Enter the number of processes:
4
Enter the burst time of the processes:
8 4 9 5
Process Burst Time Waiting Time Turnaround Time
P2 4 0 4
P4 5 4 9
P1 8 9 17
P3 9 17 26
Average Waiting Time: 7.5
Average Turnaround Time: 14.0
student@fedora:~$

Ex. No.: 6c)

PRIORITY SCHEDULING

Aim:

To implement priority scheduling technique

Algorithm:

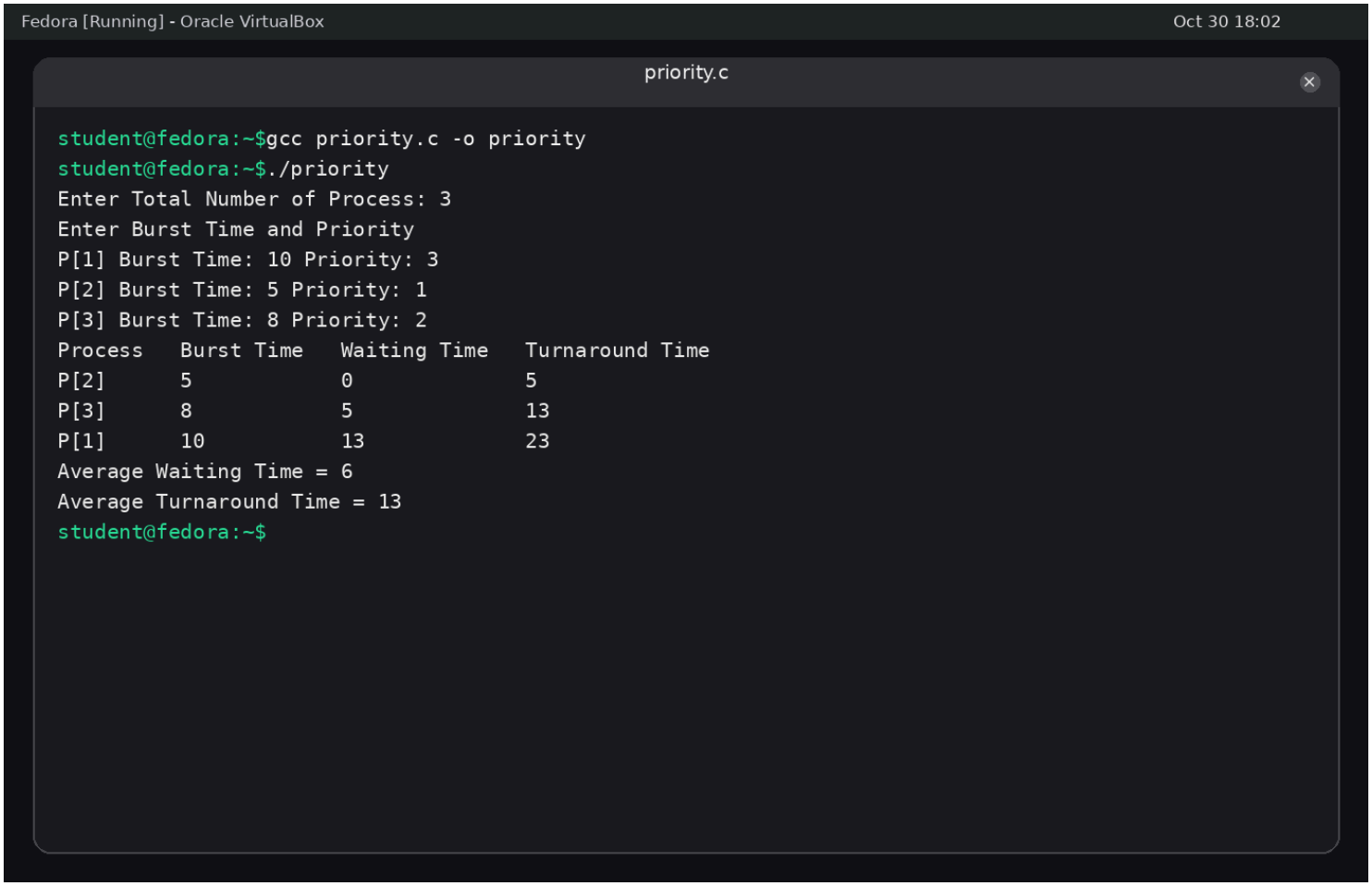
1. Get the number of processes from the user.
2. Read the process name, burst time and priority of process.
3. Sort based on burst time of all processes in ascending order based priority 4. Calculate the total waiting time and total turnaround time for each process 5. Display the process name & burst time for each process.
6. Display the total waiting time, average waiting time, turnaround time

Program Code:

```
#include<stdio.h>
int main()
{
 int bt[20],p[20],wt[20],tat[20],pr[20],i,j,n,total=0,pos,temp,avg_wt,avg_tat;
printf("Enter Total Number of Process:");
 scanf("%d",&n);

 printf("\nEnter Burst Time and Priority\n");
 for(i=0;i<n;i++)
 {
 printf("\nP[%d]\n",i+1);
 printf("Burst Time:");
 scanf("%d",&bt[i]);
 printf("Priority:");
 scanf("%d",&pr[i]);
 p[i]=i+1; //contains process number
 }
 //sorting burst time, priority and process number in ascending order using selection sort
for(i=0;i<n;i++)
 {
 pos=i;
 for(j=i+1;j<n;j++)
 {
 if(pr[j]<pr[pos])
 pos=j;
 }
 temp=pr[i];
 pr[i]=pr[pos];
 pr[pos]=temp;

 temp=bt[i];
 bt[i]=bt[pos];
 bt[pos]=temp;
 temp=p[i];
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
priority.c
student@fedora:~$gcc priority.c -o priority
student@fedora:~$./priority
Enter Total Number of Process: 3
Enter Burst Time and Priority
P[1] Burst Time: 10 Priority: 3
P[2] Burst Time: 5 Priority: 1
P[3] Burst Time: 8 Priority: 2
Process Burst Time Waiting Time Turnaround Time
P[2] 5 0 5
P[3] 8 5 13
P[1] 10 13 23
Average Waiting Time = 6
Average Turnaround Time = 13
student@fedora:~$

Ex. No.: 6d)

ROUND ROBIN SCHEDULING

Aim:

To implement the Round Robin (RR) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes and Time quantum as input from the user.
3. Read the process name, arrival time and burst time
4. Create an array **rem_bt[]** to keep track of remaining burst time of processes which is initially copy of bt[] (burst times array)
5. Create another array **wt[]** to store waiting times of processes. Initialize this array as 0. 6. Initialize time : t = 0
7. Keep traversing the all processes while all processes are not done. Do following for i'th process if it is not done yet.
   a- If rem_bt[i] > quantum
   (i) t = t + quantum
   (ii) bt_rem[i] -= quantum;
   b- Else // Last cycle for this process
   (i) t = t + bt_rem[i];
   (ii) wt[i] = t - bt[i]
   (iii) bt_rem[i] = 0; // This process is over
8. Calculate the waiting time and turnaround time for each process.
9. Calculate the average waiting time and average turnaround time.
10. Display the results.

Program Code:

```
#include<stdio.h>

int main()
{
 int i, limit, total = 0, x, counter = 0, time_quantum;
 int wait_time = 0, turnaround_time = 0, arrival_time[10], burst_time[10], temp[10];  float
average_wait_time, average_turnaround_time;
 printf("nEnter Total Number of Processes:t");
 scanf("%d", &limit);
 x = limit;
 for(i = 0; i < limit; i++)
 {
 printf("nEnter Details of Process[%d]n", i + 1);

 printf("Arrival Time:t");


 scanf("%d", &arrival_time[i]);

 printf("Burst Time:t");
```

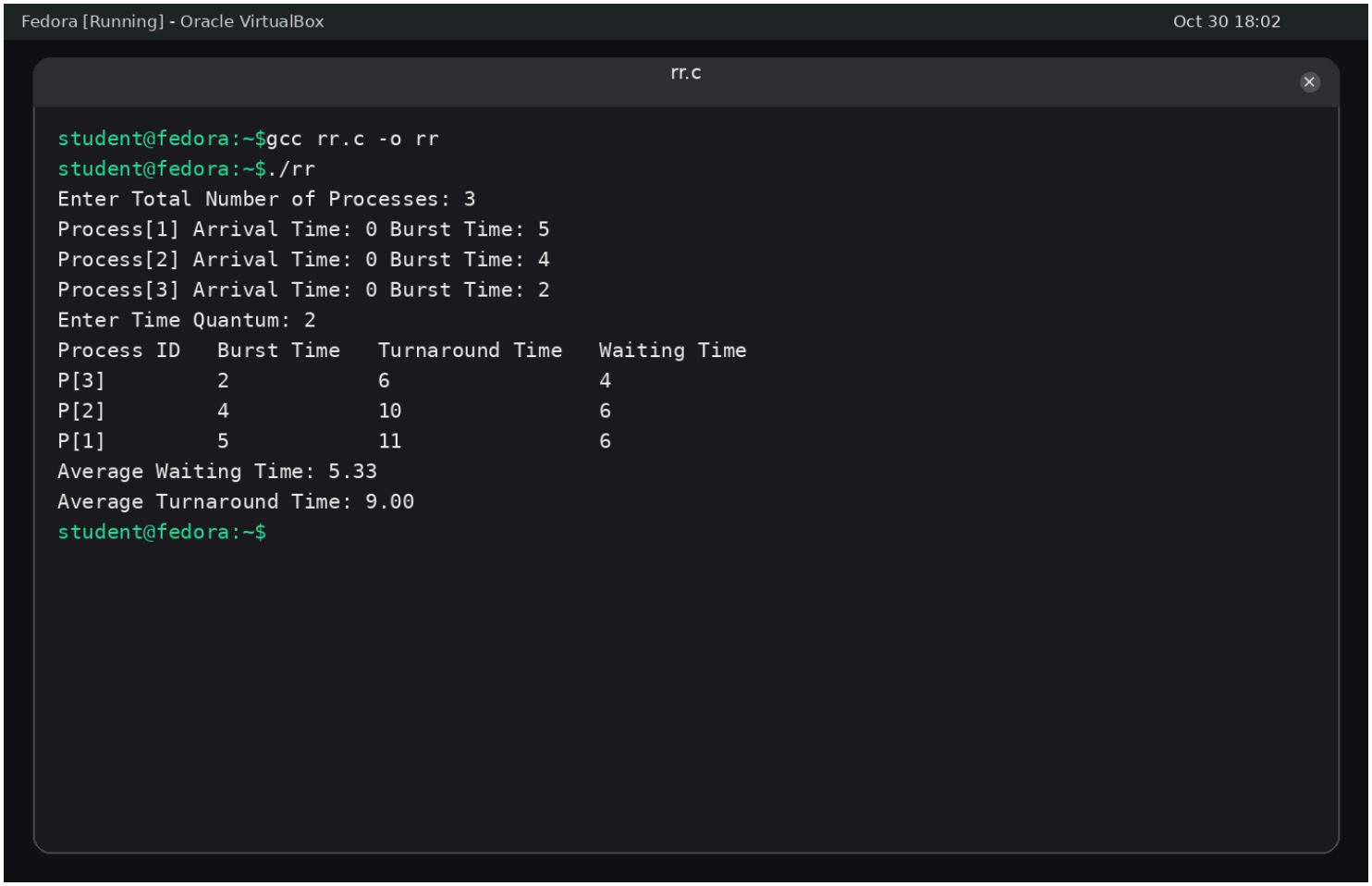
```
 scanf("%d", &burst_time[i]);

 temp[i] = burst_time[i];
 }

 printf("nEnter Time Quantum:t");
 scanf("%d", &time_quantum);
 printf("nProcess IDttBurst Timet Turnaround Timet Waiting Timen");  for(total
= 0, i = 0; x != 0;)
 {
 if(temp[i] <= time_quantum && temp[i] > 0)
 {
 total = total + temp[i];
 temp[i] = 0;
 counter = 1;
 }
 else if(temp[i] > 0)
 {
 temp[i] = temp[i] - time_quantum;
 total = total + time_quantum;
 }
 if(temp[i] == 0 && counter == 1)
 {
 x--;
 printf("nProcess[%d]tt%dtt %dttt %d", i + 1, burst_time[i], total - arrival_time[i],  total -
arrival_time[i] - burst_time[i]);
 wait_time = wait_time + total - arrival_time[i] - burst_time[i];
turnaround_time = turnaround_time + total - arrival_time[i];  counter = 0;
 }
 if(i == limit - 1)
 {
 i = 0;
 }
 else if(arrival_time[i + 1] <= total)
 {
 i++;
 }
 else
 {


 i = 0;
 }
 }

 average_wait_time = wait_time * 1.0 / limit;
 average_turnaround_time = turnaround_time * 1.0 / limit;
 printf("nnAverage Waiting Time:t%f", average_wait_time);
 printf("nAvg Turnaround Time:t%fn", average_turnaround_time);
 return 0;
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
rr.c
student@fedora:~$gcc rr.c -o rr
student@fedora:~$./rr
Enter Total Number of Processes: 3
Process[1] Arrival Time: 0 Burst Time: 5
Process[2] Arrival Time: 0 Burst Time: 4
Process[3] Arrival Time: 0 Burst Time: 2
Enter Time Quantum: 2
Process ID Burst Time Turnaround Time Waiting Time
P[3] 2 6 4
P[2] 4 10 6
P[1] 5 11 6
Average Waiting Time: 5.33
Average Turnaround Time: 9.00
student@fedora:~$

Ex. No.: 7

IPC USING SHARED MEMORY

Aim:

To write a C program to do Inter Process Communication (IPC) using shared memory between sender process and receiver process.

Algorithm:

sender

1. Set the size of the shared memory segment
2. Allocate the shared memory segment using shmget
3. Attach the shared memory segment using shmat
4. Write a string to the shared memory segment using sprintf
5. Set delay using sleep
6. Detach shared memory segment using shmdt

receiver

1. Set the size of the shared memory segment
2. Allocate the shared memory segment using shmget
3. Attach the shared memory segment using shmat
4. Print the shared memory contents sent by the sender process.
5. Detach shared memory segment using shmdt

Program Code:

sender.c

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define SharedMemSize 50
void main()
{
 char c;
 int shmid;
 key_t key;


 char *shared_memory;
```

```
 key = 5677;
 //Create segment with the key specified
 if ((shmid = shmget(key, SharedMemSize, IPC_CREAT |
0666)) < 0)
 {
          //perror explains error code
 perror("shmget");
 exit(1);
 }

 //Attach the segment
 if((shared_memory= shmat(shmid, NULL, 0)) == (char *) -1)   {
 perror("shmat");
 exit(1);
 }

 sprintf(shared_memory," Welcome to Shared Memory");
sleep(2);
 exit(0);
 }
```

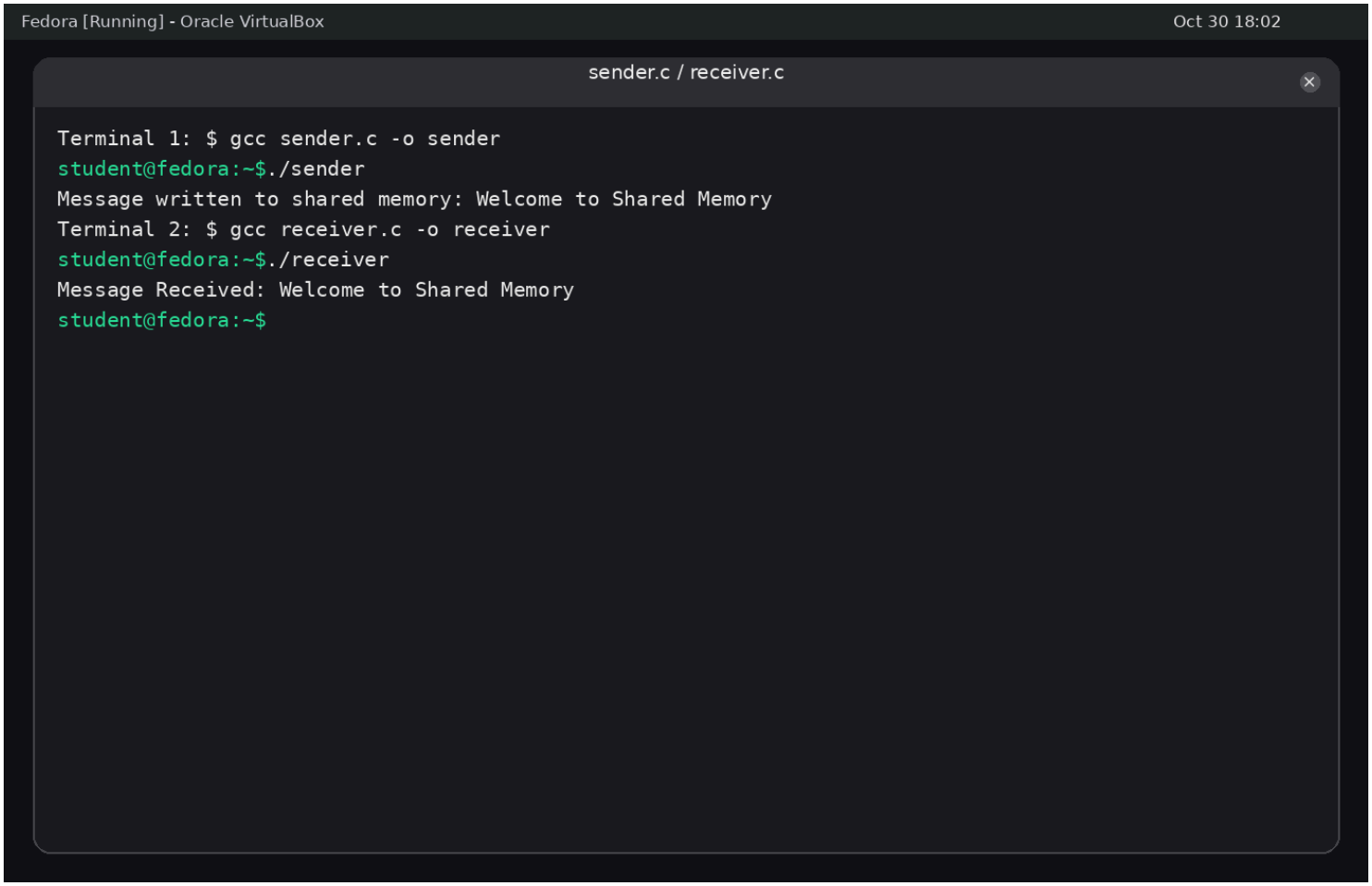
receiver.c

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#define SharedMemSize 50

void main()
{
 int shmid;
 key_t key;
 char *shared_memory;
 key = 5677;
 if ((shmid = shmget(key, SharedMemSize, 0666)) < 0)   {
 perror("shmget");
 exit(1);


 }

 //Attach the segment to our data space
 if((shared_memory = shmat(shmid, NULL, 0))==(char *) -1)   {
 perror("shmat");
 exit(1);
 }
 //Read the message sender sent to the shared memory
printf("Message Received: %s \n",shared_memory);  exit(0);
 }
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
sender.c / receiver.c
Terminal 1: $ gcc sender.c -o sender
student@fedora:~$./sender
Message written to shared memory: Welcome to Shared Memory
Terminal 2: $ gcc receiver.c -o receiver
student@fedora:~$./receiver
Message Received: Welcome to Shared Memory
student@fedora:~$

Ex. No.: 8

PRODUCER CONSUMER USING SEMAPHORES

Aim: To write a program to implement solution to producer consumer problem using semaphores.

Algorithm:

1. Initialize semaphore empty, full and mutex.
2. Create two threads- producer thread and consumer thread.
3. Wait for target thread termination.
4. Call sem_wait on empty semaphore followed by mutex semaphore before entry into critical section.
5. Produce/Consume the item in critical section.
6. Call sem_post on mutex semaphore followed by full semaphore
7. before exiting critical section.
8. Allow the other thread to enter its critical section.
9. Terminate after looping ten times in producer and consumer Threads each.

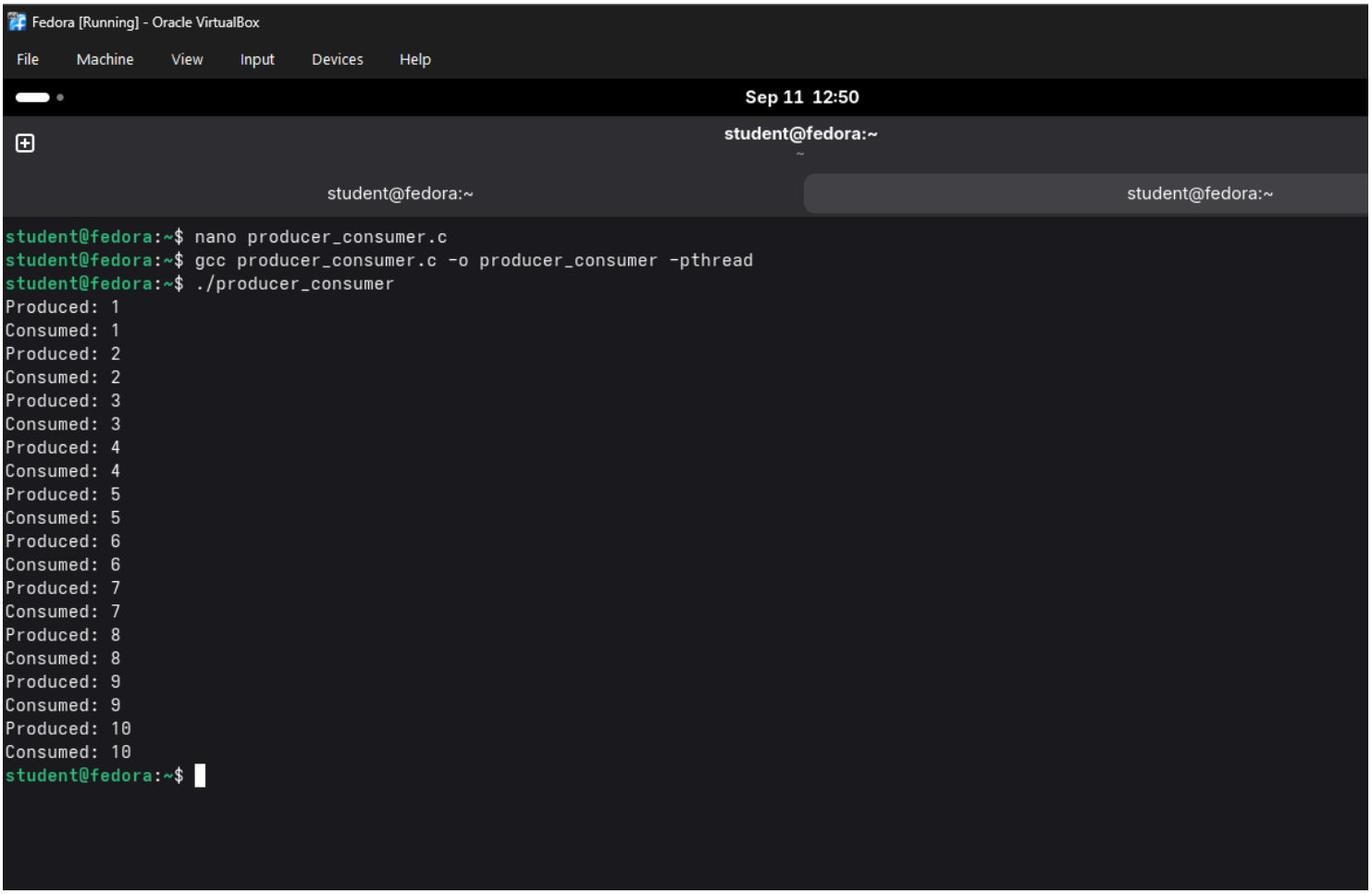
Program Code:

```
#include<stdio.h>
#include<stdlib.h>

int mutex=1,full=0,empty=3,x=0;

int main()
{
        int n;
        void producer();
        void consumer();
        int wait(int);
        int signal(int);
        printf("\n1.Producer\n2.Consumer\n3.Exit");
        while(1)
        {
                printf("\nEnter your choice:");
                scanf("%d",&n);
                switch(n)
                {
                        case 1: if((mutex==1)&&(empty!=0))
                                                producer();
                                else
                                                printf("Buffer is full!!");


                                break;
                        case 2: if((mutex==1)&&(full!=0))
                                                consumer();
                                else
                                                printf("Buffer is empty!!");
```


Fedora [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Sep 11 12:50
student@fedora:~
student@fedora:~
student@fedora:~
student@fedora:~$ nano producer_consumer.c
student@fedora:~$ gcc producer_consumer.c -o producer_consumer -pthread
student@fedora:~$ ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
student@fedora:~$

Ex. No.: 9

DEADLOCK AVOIDANCE

Aim:

To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i 2.
Find an i such that both:
finish[i]=false and $Need_i$<= work
3. If no such i exists go to step 6
4. Compute work=work+allocationi
5. Assign finish[i] to true and go to step 2
6. If finish[i]==true for all i, then print safe sequence
7. Else print there is no safe sequence

Program Code:

```
#include <stdio.h>
int main()
{
 // P0, P1, P2, P3, P4 are the Process names here

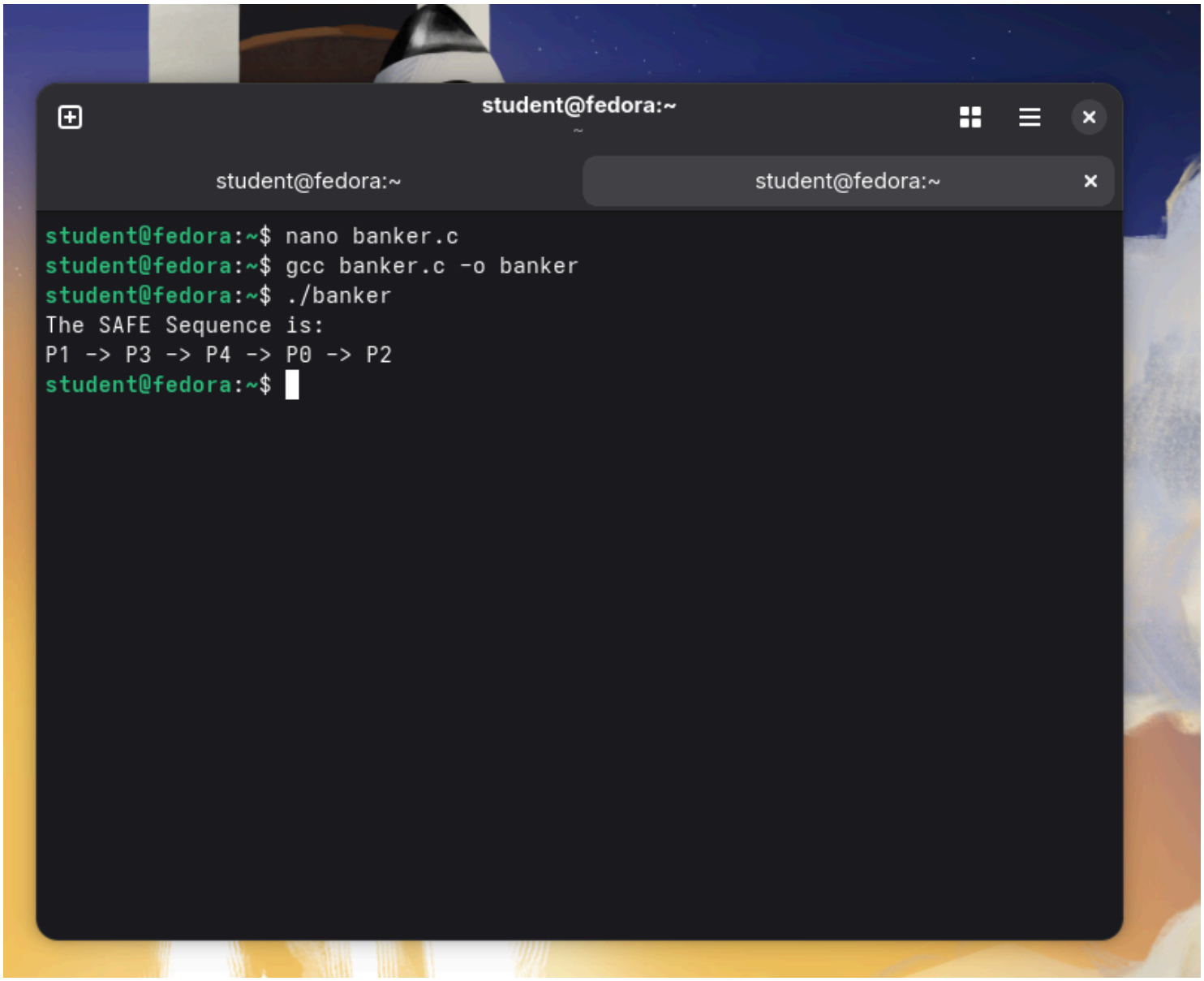
 int n, m, i, j, k;
 n = 5; // Number of processes
 m = 3; // Number of resources
 int alloc[5][3] = { { 0, 1, 0 }, // P0 // Allocation Matrix
 { 2, 0, 0 }, // P1
 { 3, 0, 2 }, // P2
 { 2, 1, 1 }, // P3
 { 0, 0, 2 } }; // P4

 int max[5][3] = { { 7, 5, 3 }, // P0 // MAX Matrix
 { 3, 2, 2 }, // P1
 { 9, 0, 2 }, // P2
 { 2, 2, 2 }, // P3
 { 4, 3, 3 } }; // P4

 int avail[3] = { 3, 3, 2 }; // Available Resources

 int f[n], ans[n], ind = 0;
 for (k = 0; k < n; k++) {
 f[k] = 0;
 }


 int need[n][m];
 for (i = 0; i < n; i++) {
 for (j = 0; j < m; j++)
```


student@fedora:~
student@fedora:~
student@fedora:~
student@fedora:~$ nano banker.c
student@fedora:~$ gcc banker.c -o banker
student@fedora:~$ ./banker
The SAFE Sequence is:
P1 -> P3 -> P4 -> P0 -> P2
student@fedora:~$

Ex. No.: 10a)

BEST FIT

Aim:

To implement Best Fit memory allocation technique using Python.

Algorithm:

1. Input memory blocks and processes with sizes
2. Initialize all memory blocks as free.
3. Start by picking each process and find the minimum block size that can be assigned to current process
4. If found then assign it to the current process.
5. If not found then leave that process and keep checking the further processes.

Program Code:

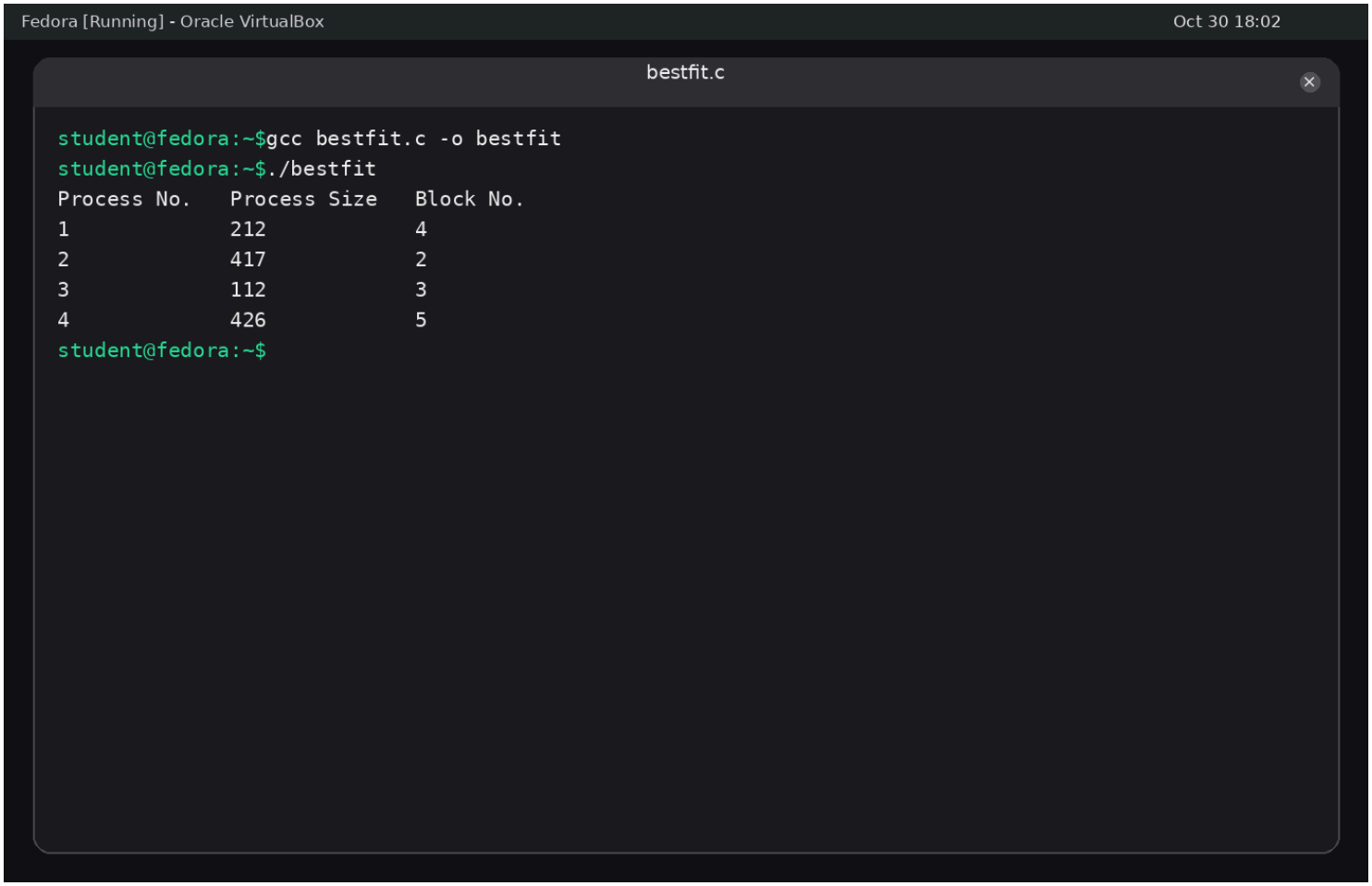
```
#include<stdio.h>
#include<string.h>
void bestFit(int blockSize[], int m, int processSize[], int n)
{
 // Stores block id of the block allocated to a
 // process
 int allocation[n];

 // Initially no block is assigned to any process
 memset(allocation, -1, sizeof(allocation));

 // pick each process and find suitable blocks
 // according to its size ad assign to it
 for (int i=0; i<n; i++)
 {
 // Find the best fit block for current process
 int bestIdx = -1;
 for (int j=0; j<m; j++)
 {
 if (blockSize[j] >= processSize[i])
 {
 if (bestIdx == -1)
 bestIdx = j;
 else if (blockSize[bestIdx] > blockSize[j])
 bestIdx = j;
 }
 }



 // If we could find a block for current process  if
(bestIdx != -1)
 {
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
bestfit.c
student@fedora:~$gcc bestfit.c -o bestfit
student@fedora:~$./bestfit
Process No. Process Size Block No.
1 212 4
2 417 2
3 112 3
4 426 5
student@fedora:~$

Ex. No.: 10b)

FIRST FIT

Aim:

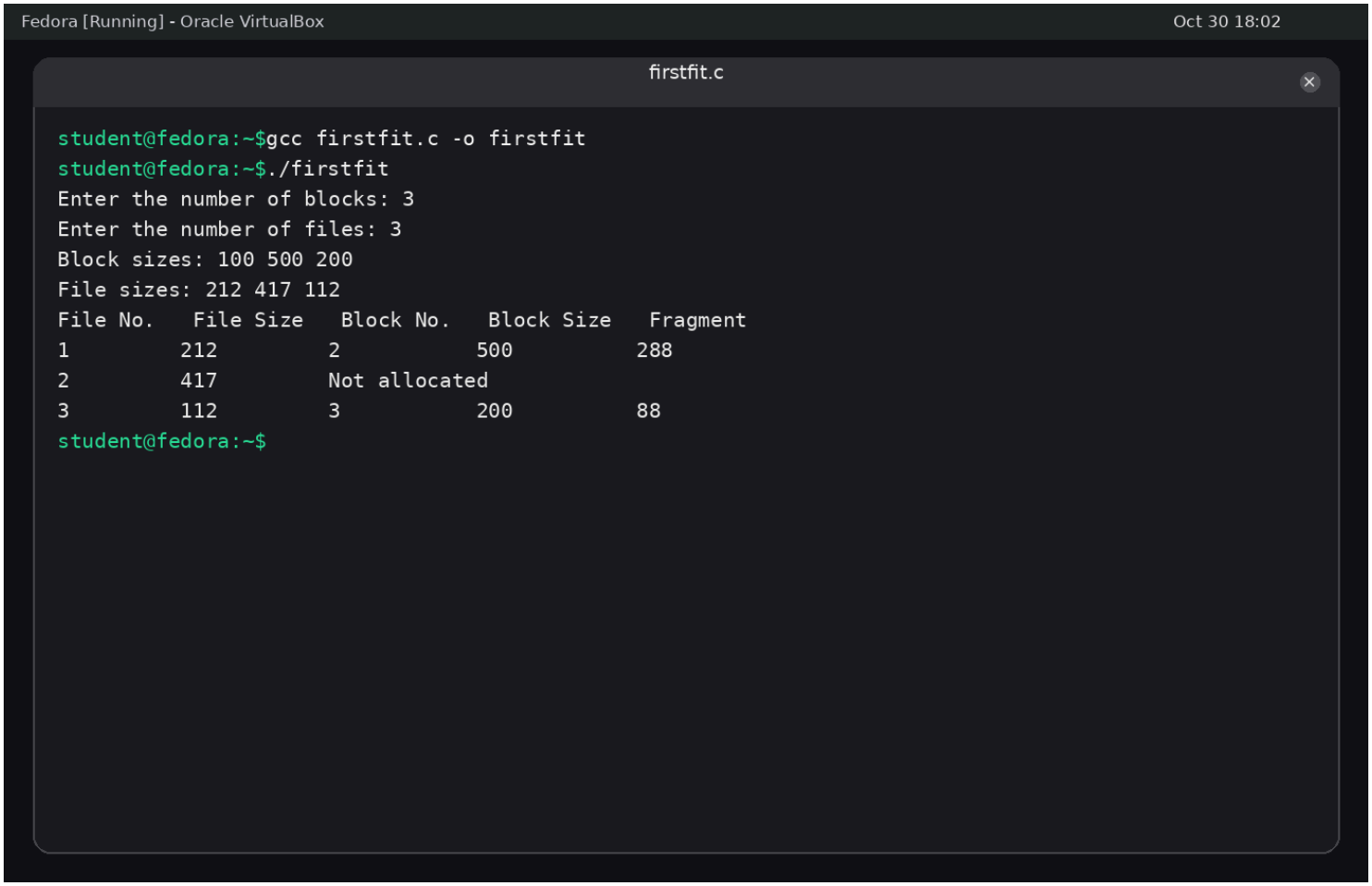
To write a C program for implementation memory allocation methods for fixed partition using first fit.

Algorithm:

1. Define the max as 25.
2: Declare the variable frag[max],b[max],f[max],i,j,nb,nf,temp, highest=0, bf[max],ff[max]. 3: Get the number of blocks,files,size of the blocks using for loop.
4: In for loop check bf[j]!=1, if so temp=b[j]-f[i]
5: Check highest

Program Code:

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp;
static int bf[max],ff[max];
clrscr();
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files:-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
{
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
firstfit.c
student@fedora:~$gcc firstfit.c -o firstfit
student@fedora:~$./firstfit
Enter the number of blocks: 3
Enter the number of files: 3
Block sizes: 100 500 200
File sizes: 212 417 112
File No. File Size Block No. Block Size Fragment
1 212 2 500 288
2 417 Not allocated
3 112 3 200 88
student@fedora:~$

Ex. No.: 11a)

FIFO PAGE REPLACEMENT

Aim:

To find out the number of page faults that occur using First-in First-out(FIFO) page replacement technique.

Algorithm:

1. Declare the size with respect to page length
2. Check the need of replacement from the page to memory
3. Check the need of replacement from old page to new page in memory 4.
Form a queue to hold all pages
5. Insert the page require memory into the queue
6. Check for bad replacement and page fault
7. Get the number of processes to be inserted
8. Display the values

Program Code:

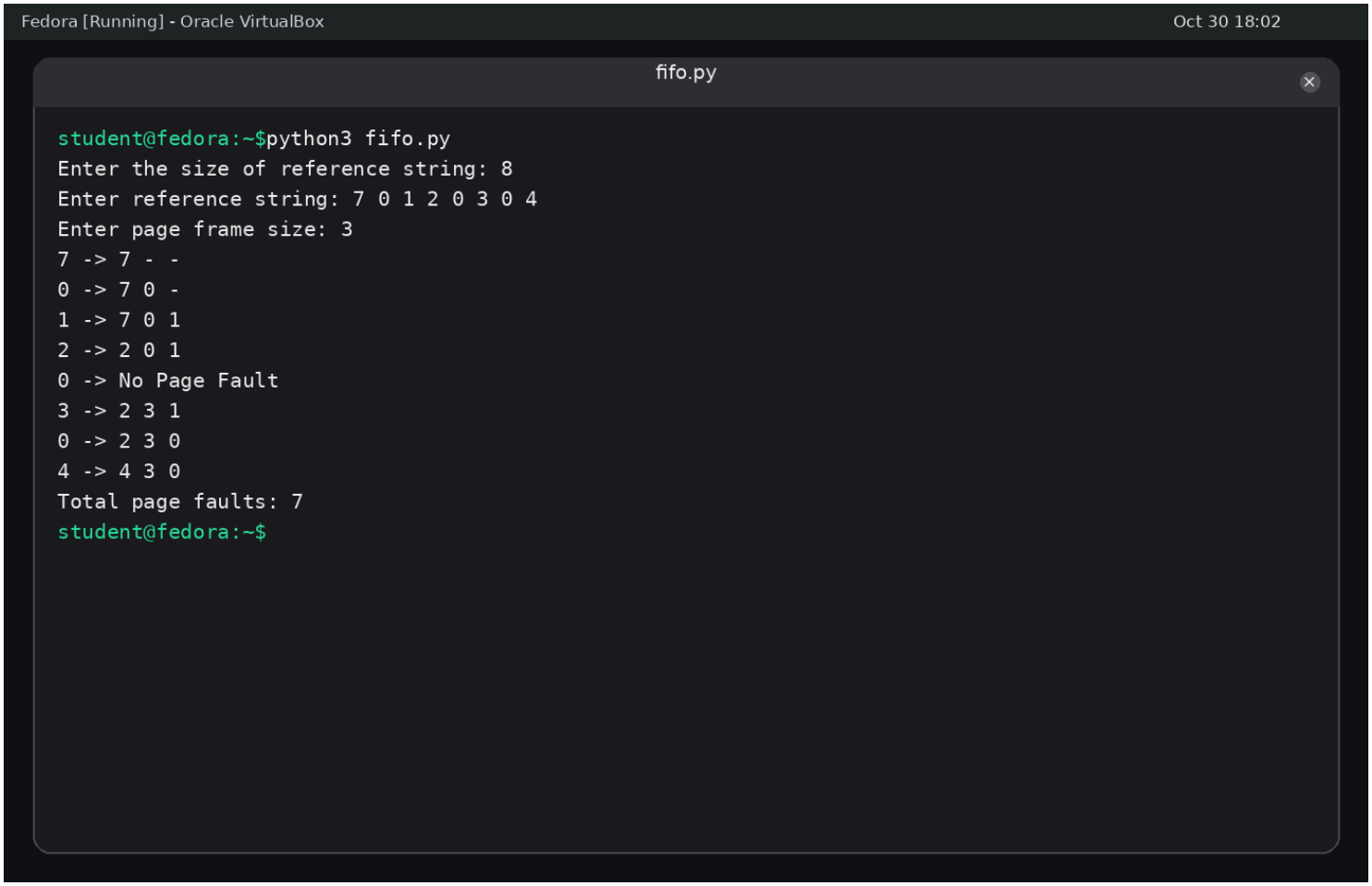
```
def fifo():
 global a,n,m
 f = -1
 page_faults = 0
 page = []
 for i in range(m):
 page.append(-1)

 for i in range(n):
 flag = 0
 for j in range(m):
 if(page[j] == a[i]):
 flag = 1
 break

 if flag == 0:
 f=(f+1)%m
 page[f] = a[i]
 page_faults+=1
 print "\n%d ->" % (a[i]),
 for j in range(m):
 if page[j] != -1:
 print page[j],
 else:


 print "-",
 else:
 print "\n%d -> No Page Fault" % (a[i]),
 print "\n Total page faults : %d." % (page_faults)

a = []
n = input("\n Enter the size of reference string : ")
 for i in range(n):
        a.append(input(" Enter [%2d] : " %
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
fifo.py
student@fedora:~$python3 fifo.py
Enter the size of reference string: 8
Enter reference string: 7 0 1 2 0 3 0 4
Enter page frame size: 3
7 -> 7 - -
0 -> 7 0 -
1 -> 7 0 1
2 -> 2 0 1
0 -> No Page Fault
3 -> 2 3 1
0 -> 2 3 0
4 -> 4 3 0
Total page faults: 7
student@fedora:~$

Ex. No.: 11b)

LRU

Aim:

To write a c program to implement LRU page replacement algorithm.

Algorithm:

1: Start the process
2: Declare the size
3: Get the number of pages to be inserted
4: Get the value
5: Declare counter and stack
6: Select the least recently used page by counter value
7: Stack them according the selection.
8: Display the values
9: Stop the process

Program Code:

```
#include<stdio.h>

int findLRU(int time[], int n){
        int i, minimum = time[0], pos = 0;

        for(i = 1; i < n; ++i){
                if(time[i] < minimum){
                        minimum = time[i];
                        pos = i;
                }
        }

        return pos;
}

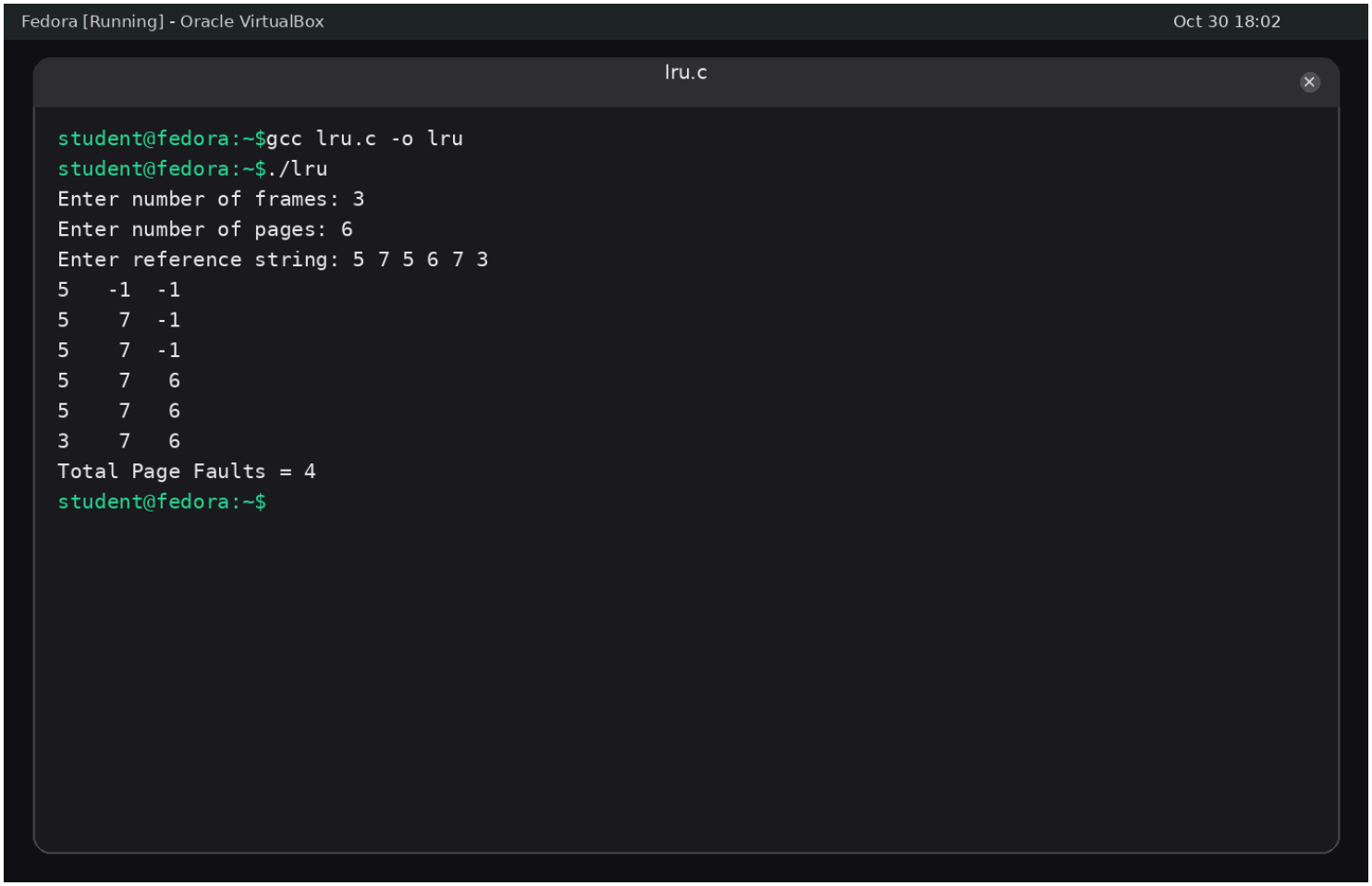
int main()
{
 int no_of_frames, no_of_pages, frames[10], pages[30], counter = 0, time[10], flag1, flag2, i, j,  pos, faults
= 0;
        printf("Enter number of frames: ");
        scanf("%d", &no_of_frames);

        printf("Enter number of pages: ");
        scanf("%d", &no_of_pages);

        printf("Enter reference string: ");


 for(i = 0; i < no_of_pages; ++i){
                scanf("%d", &pages[i]);
 }

        for(i = 0; i < no_of_frames; ++i){
 frames[i] = -1;
 }
 for(i = 0; i < no_of_pages; ++i){
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
lru.c
student@fedora:~$gcc lru.c -o lru
student@fedora:~$./lru
Enter number of frames: 3
Enter number of pages: 6
Enter reference string: 5 7 5 6 7 3
5 -1 -1
5 7 -1
5 7 -1
5 7 6
5 7 6
3 7 6
Total Page Faults = 4
student@fedora:~$

Ex. No.: 11c)

Optimal

Aim:
To write a c program to implement Optimal page replacement algorithm.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least frequently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int i,j,nof,nor,flag=0,ref[50],frm[50],pf=0,victim=-1; int recent[10],optcal[50],count=0;
int optvictim(); void main()
{ clrscr();
printf("\n OPTIMAL PAGE REPLACEMENT
ALGORITHN");
printf("\n..... ");
.....
printf("\nEnter the no.of frames");
scanf("%d",&nof);
printf("Enter the no.of reference string");

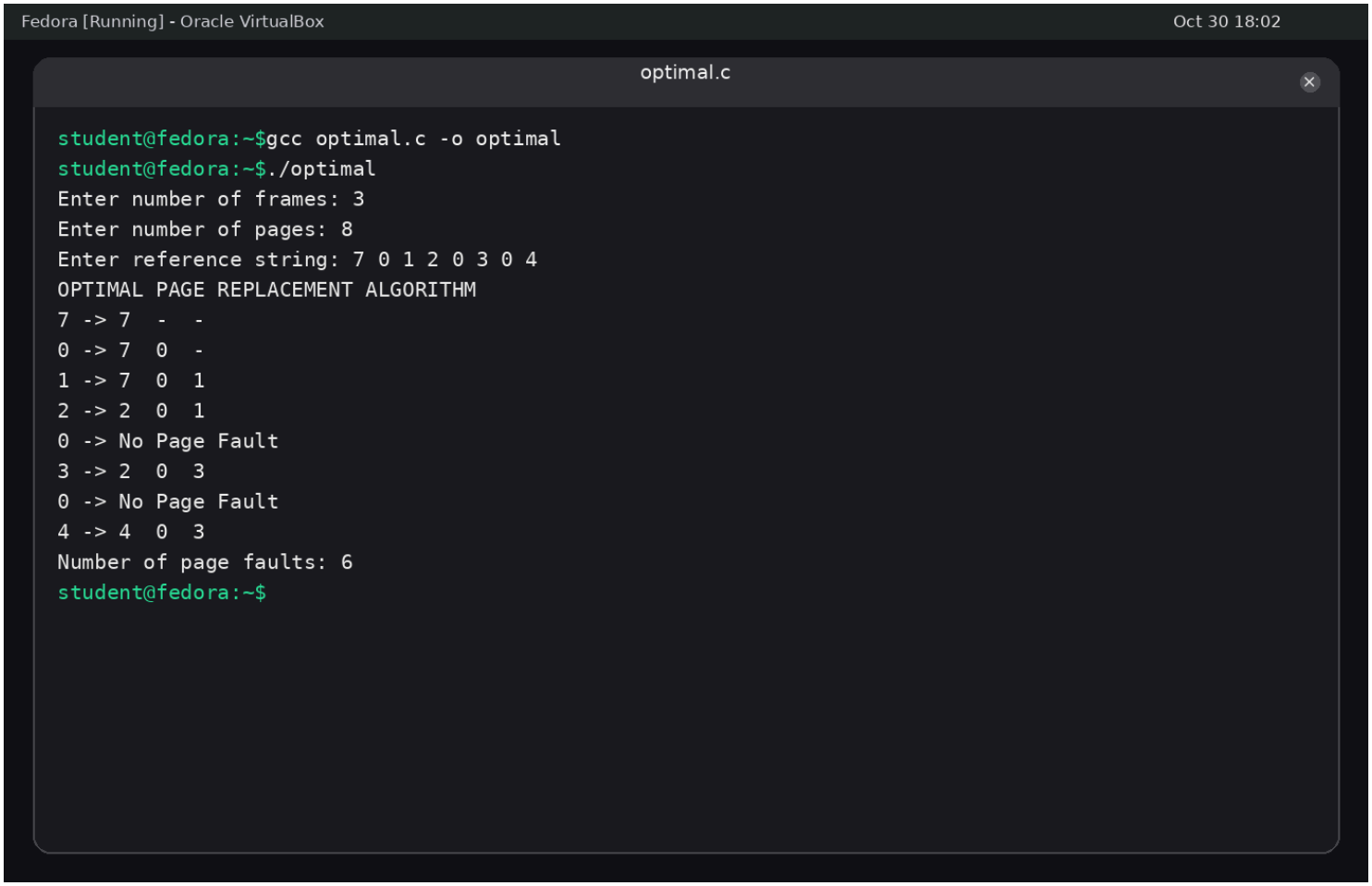
scanf("%d",&nor);
printf("Enter the reference string");
for(i=0;i<nor;i++)
scanf("%d",&ref[i]);
clrscr();
printf("\n OPTIMAL PAGE REPLACEMENT ALGORITHM");
printf("\n.....");
printf("\nThe given string");
printf("\n.....\n");
for(i=0;i<nor;i++)
printf("%4d",ref[i]);
for(i=0;i<nof;i++)
{
```

```
frm[i]=-1;
optcal[i]=0;
}
for(i=0;i<10;i++)
recent[i]=0;
printf("\n");
for(i=0;i<nor;i++)
{
flag=0;
printf("\n\tref no %d ->\t",ref[i]);
for(j=0;j<nof;j++)
{
if(frm[j]==ref[i])

{
flag=1;
break;
}
}
if(flag==0)
{
count++;
if(count<=nof)
victim++; else
victim=optvictim(i);
pf++;
frm[victim]=ref[i];
for(j=0;j<nof;j++)
printf("%4d",frm[j]);
}
}

printf("\n Number of page faults: %d",pf); getch();
}
int optvictim(int index)
{
int i,j,temp,notfound; for(i=0;i<nof;i++)
{
notfound=1;

for(j=index;j<nor;j++)
if(frm[i]==ref[j])
{
notfound=0;
optcal[i]=j;
break;
}
if(notfound==1) return i;
}
temp=optcal[0];
for(i=1;i<nof;i++)
if(temp<optcal[i])
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
optimal.c
student@fedora:~$gcc optimal.c -o optimal
student@fedora:~$./optimal
Enter number of frames: 3
Enter number of pages: 8
Enter reference string: 7 0 1 2 0 3 0 4
OPTIMAL PAGE REPLACEMENT ALGORITHM
7 -> 7 - -
0 -> 7 0 -
1 -> 7 0 1
2 -> 2 0 1
0 -> No Page Fault
3 -> 2 0 3
0 -> No Page Fault
4 -> 4 0 3
Number of page faults: 6
student@fedora:~$

Ex. No.: 12

File Organization Technique- Single and Two level directory

AIM:

To implement File Organization Structures in C are

a. Single Level Directory
b. Two-Level Directory
c. Hierarchical Directory Structure
d. Directed Acyclic Graph Structure

a. Single Level

Directory

ALGORITHM

1. Start
2. Declare the number, names and size of the directories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories.
5. Stop.

PROGRAM:

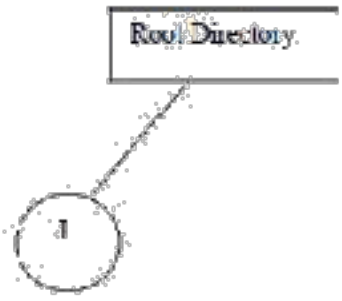
```
#include<stdio.h>
#include<stdlib.h>
#include<graphics.h>
void main()
{
int gd=DETECT,gm,count,i,j,mid,cir_x;
char fname[10][20];
initgraph(&gd,&gm,"c:\\tc\\bgi");
cleardevice();
setbkcolor(Green);
puts("Enter the number of files");
scanf("%d",&count);
for(i=0;i<count;i++)
{
```

```
  cleardevice();
  setbkcolor(GREEN);
  printf("Enter the file %d name",i+1);
  scanf("%s",fname[i]);

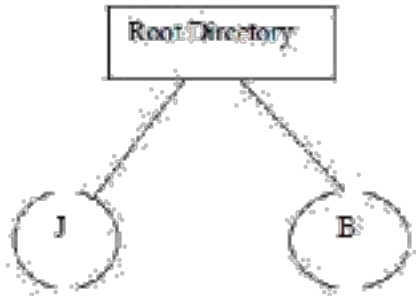
  setfillstyle(1,MAGENTA);
  mid=640/count; cir_x=mid/3;
  bar3d(270,100,370,150,0,0);
  settextstyle(2,0,4);
  settextjustify(1,1);
  outtextxy(320,125,"Root Directory");
  setcolor(BLUE);
  for(j=0;j<=i;j++,cir_x+=mid)
  {
  line(320,150,cir_x,250);
  fillellipse(cir_x,250,30,30);
  outtextxy(cir_x,250,fname[j]);
  }
 }
}
```

OUTPUT:

Enter the Number of files
2
Enter the file1 J



Enter the file2 B



b. Two-level directory Structure

ALGORITHM:

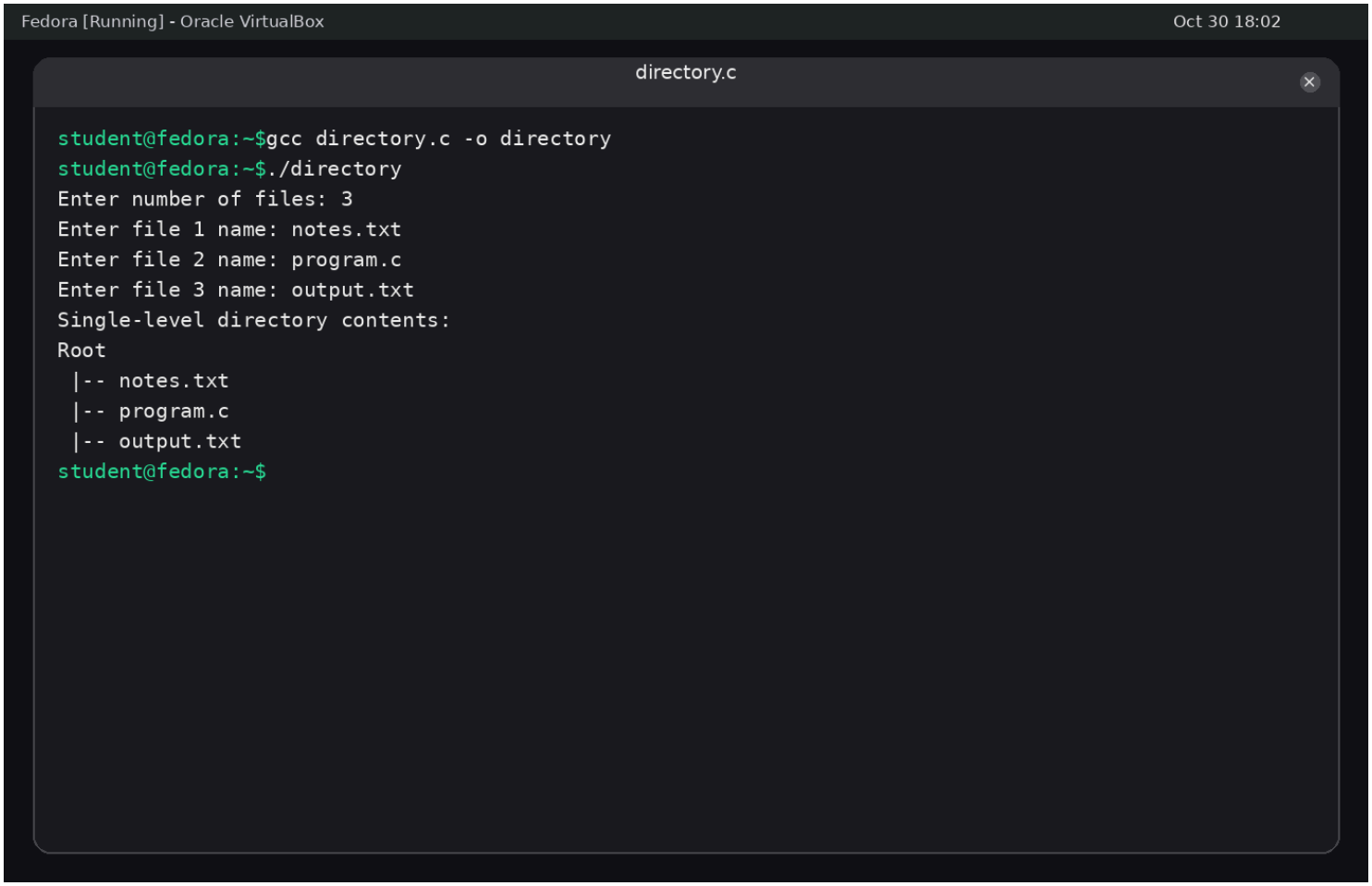
1. Start
2. Declare the number, names and size of the directories and subdirectories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories and subdirectories.
5. Stop.

PROGRAM:

```
#include<stdio.h>
#include<graphics.h>
struct tree_element
{
char name[20];
int x,y,ftype,lx,rx,nc,level; struct tree_element *link[5];
}; typedef struct tree_element node;
void main() {
int gd=DETECT,gm; node *root;
root = NULL; clrscr();
create(&root,0,"null",0,630,320);
clrscr();
initgraph(&gd,&gm,"c:\\tc\\bgi");
display(root);
getch();
closegraph();
}
create(node **root,int lev,char *dname,int lx,int rx,int x)
{
int i,gap;
if(*root==NULL)
{
(*root)=(node*)malloc(sizeof(node));
printf("enter name of dir/file(under %s):",dname);fflush(stdin);
gets((*root)->name);
if(lev==0||lev==1)
(*root)->ftype=1;
```

```
else

(*root)->ftype=2;
(*root)->level=lev;
(*root)->y=50+lev*50;
(*root)->x=x;
(*root)->lx=lx;
(*root)->rx=rx;
for(i=0;i<5;i++)
(*root)->link[i]=NULL;
if((*root)->ftype==1)
{
if(lev==0||lev==1)
{
if((*root)->level==0)
printf("How many users");
else
printf("How many files");
printf("(for%s):",(*root)->name);
scanf("%d",&(*root)->nc);
}
else(*root)->nc=0;
if((*root)->nc==0)
gap=rx-lx;
else
gap=(rx-lx)/(*root)->nc;
for(i=0;i<(*root)->nc;i++)
create(&((*root)->link[i]),lev+1,(*root)->name,lx+gap*i,lx+gap*i+gap,lx+gap*i+gap/2);
}
else
(*root)->nc=0;
}
}
display(node *root)
{
int i;
settextstyle(2,0,4);
settextjustify(1,1);
setfillstyle(1,BLUE);
setcolor(14);
if(root!=NULL)
```


Fedora [Running] - Oracle VirtualBox
Oct 30 18:02
directory.c
student@fedora:~$gcc directory.c -o directory
student@fedora:~$./directory
Enter number of files: 3
Enter file 1 name: notes.txt
Enter file 2 name: program.c
Enter file 3 name: output.txt
Single-level directory contents:
Root
|-- notes.txt
|-- program.c
|-- output.txt
student@fedora:~$